

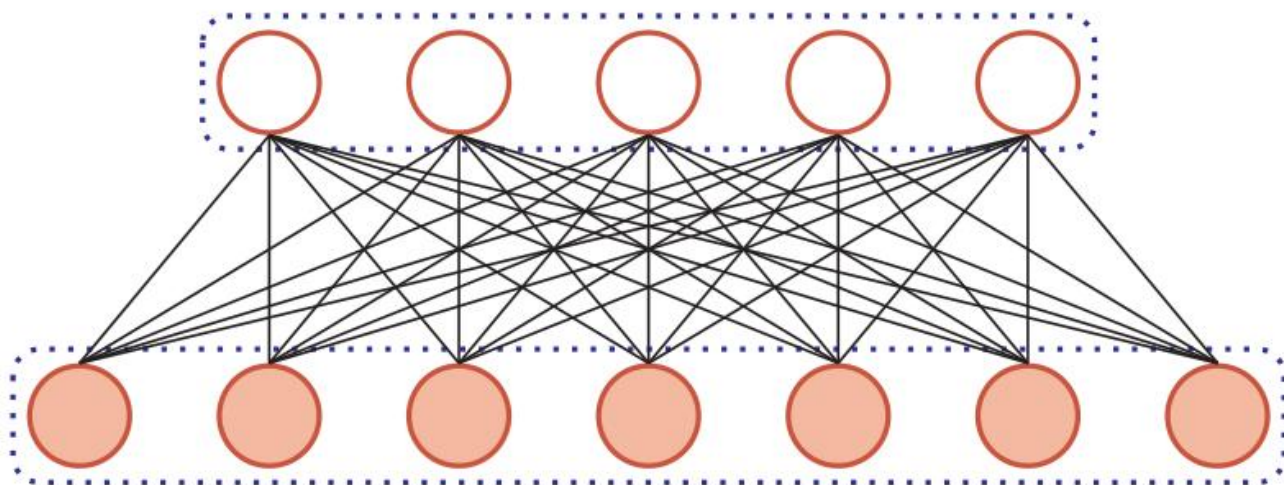
《神经网络与深度学习》

卷积神经网络

<https://nndl.github.io/>

全连接前馈神经网络

► 权重矩阵的参数非常多



► 局部不变性特征

- 自然图像中的物体都具有局部不变性特征
 - 尺度缩放、平移、旋转等操作不影响其语义信息。
- 全连接前馈网络很难提取这些局部不变特征

卷积神经网络

► 卷积神经网络 (Convolutional Neural Networks, CNN)

- 一种前馈神经网络
- 受生物学上感受野 (Receptive Field) 的机制而提出的
 - 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。

► 卷积神经网络有三个结构上的特性：

- 局部连接
- 权重共享
- 空间或时间上的次采样

卷积定义

► 一般定义函数 f, g 的卷积 $f * g(n)$ 如下:

► 连续形式:

$$f * g(n) = \int_{-\infty}^{+\infty} f(\tau)g(n - \tau)d\tau$$

► 离散形式:

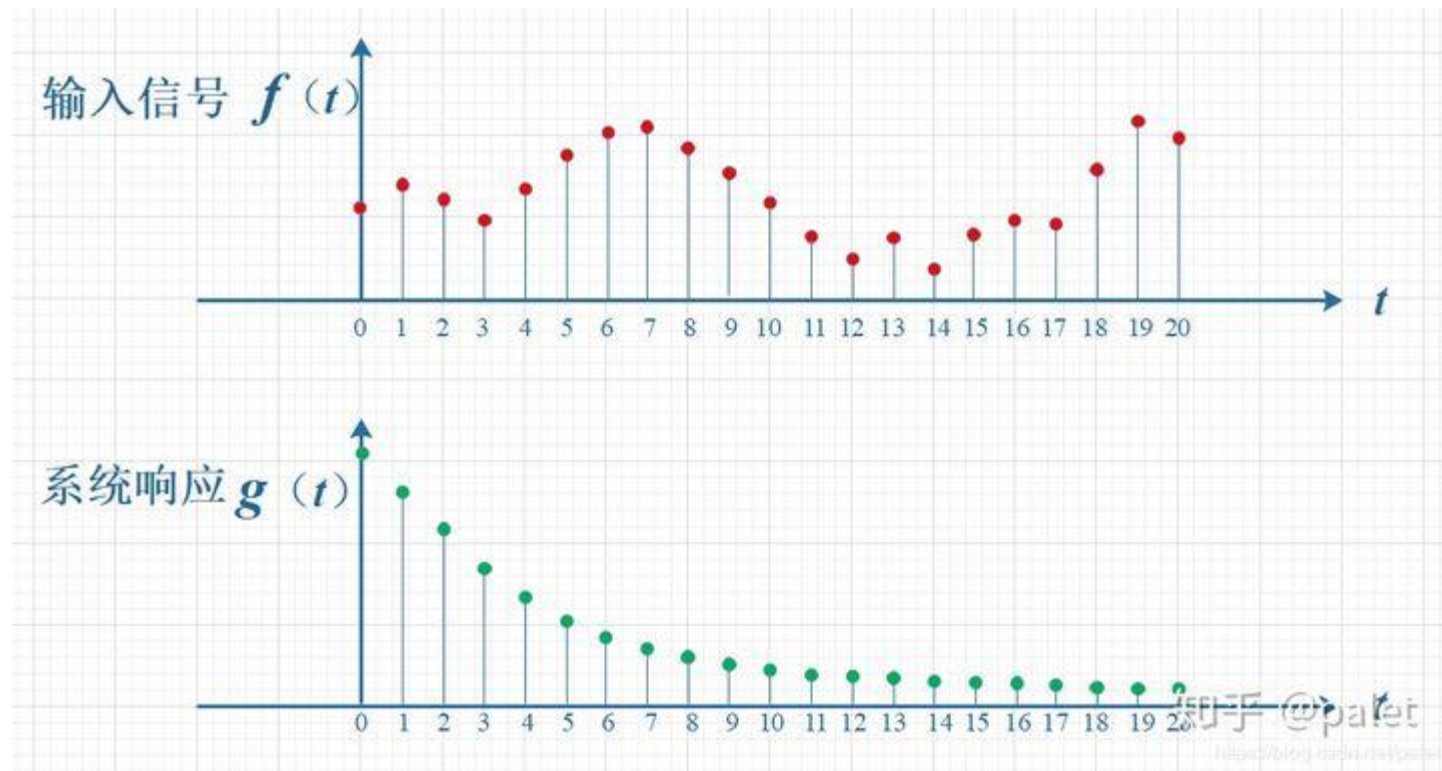
$$f * g(n) = \sum_{\tau=-\infty}^{+\infty} f(\tau)g(n - \tau)$$

卷: 翻转-滑动

积: 积分/加权求和。

直观理解

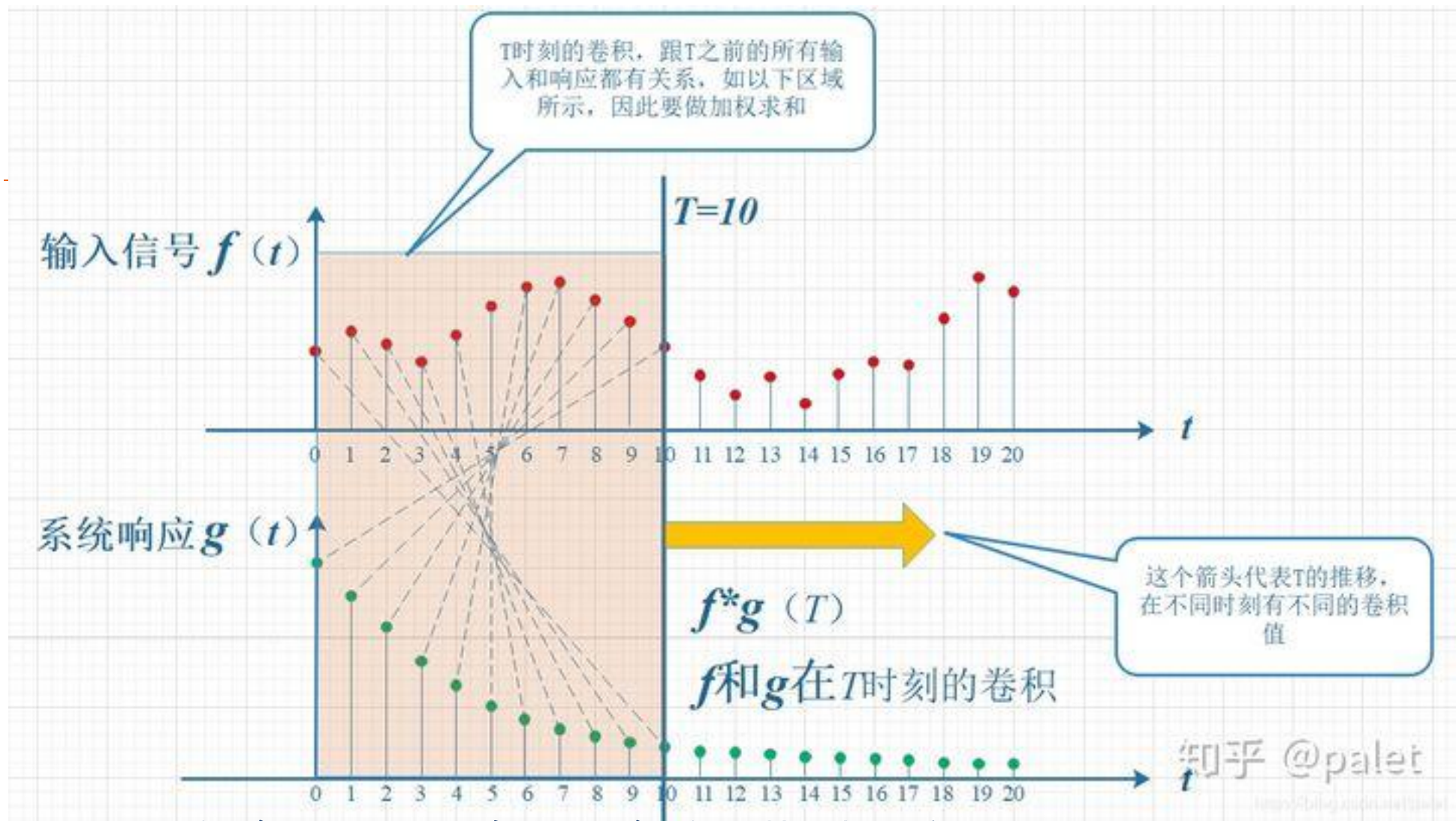
输入信号是 $f(t)$ ，是随时间变化的。系统响应函数是 $g(t)$ ，图中的响应函数是随时间指数下降的，它的物理意义是说：如果在 $t=0$ 的时刻有一个输入，那么随着时间的流逝，这个输入将不断衰减。换言之，到了 $t=T$ 时刻，原来在 $t=0$ 时刻的输入 $f(0)$ 的值将衰减为 $f(0)g(T)$ 。



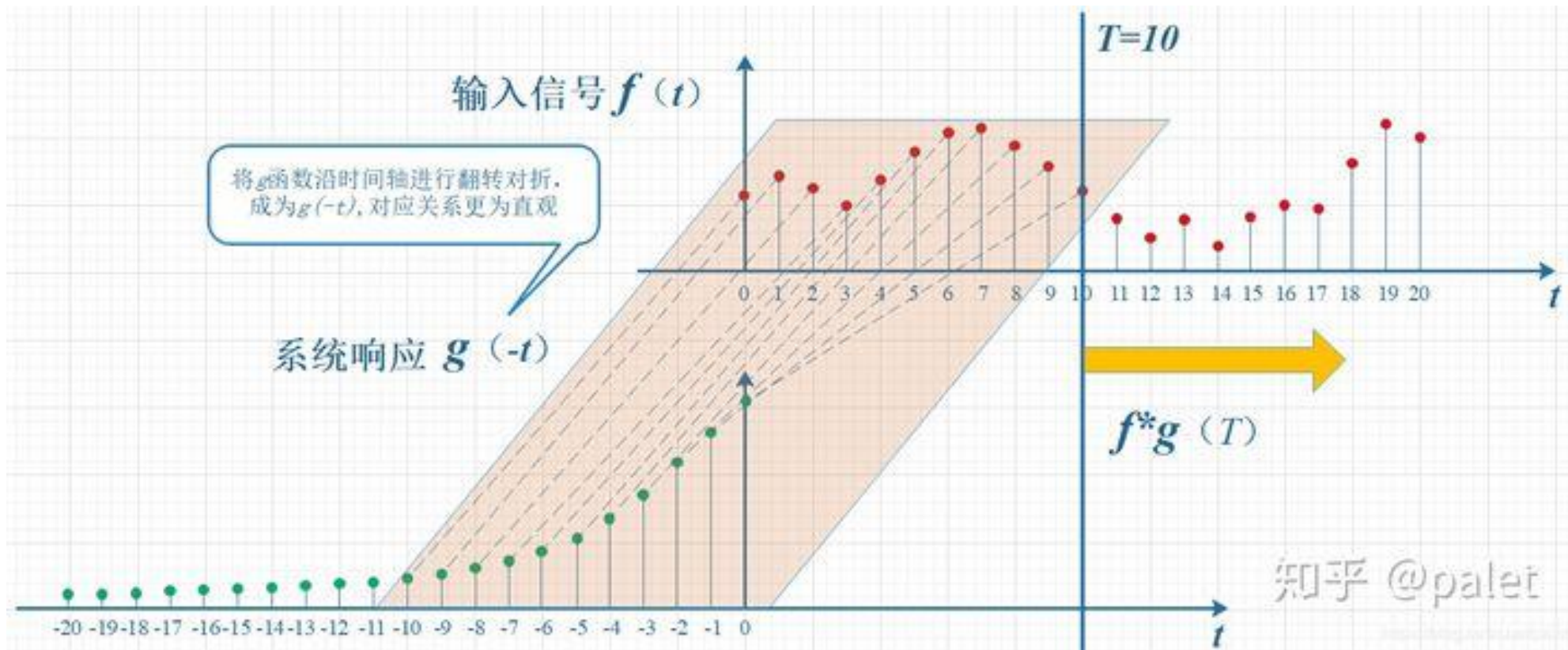
<https://www.zhihu.com/question/22298352/answer/637156871> 来源：知乎著作权归作者所有。

直观理解

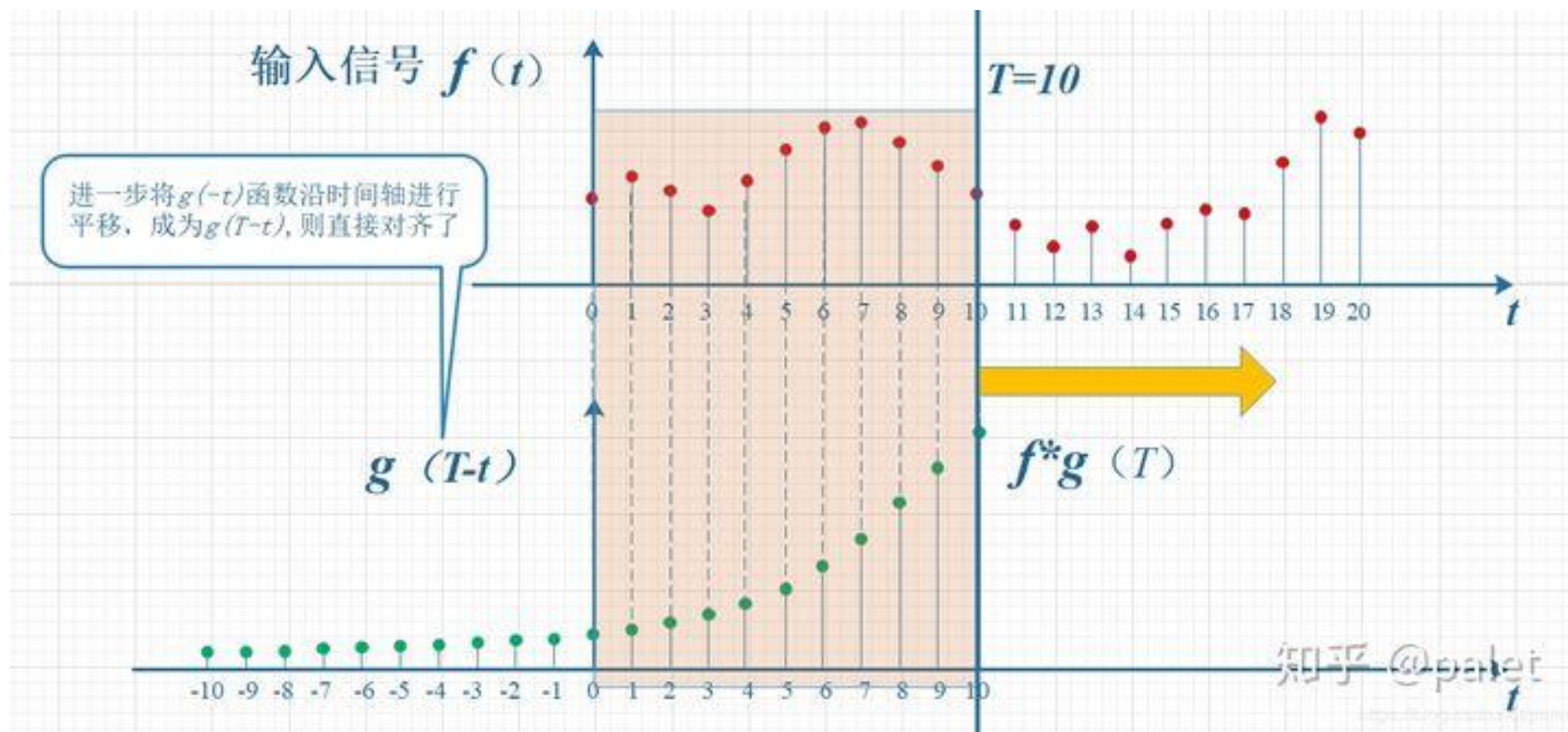
连续输入，每个时刻都有新的信号进来，最终输出的是所有之前输入信号的累积效果。



在 $T=10$ 时刻，输出结果跟图中带标记的区域整体有关。 $f(10)$ 因为是刚输入的，所以其输出结果应该是 $f(10)g(0)$ ，而时刻 $t=9$ 的输入 $f(9)$ ，只经过了1个时间单位的衰减，所以产生的输出应该是 $f(9)g(1)$ 。



第一步：翻转



第二步：滑动

卷积

- ▶ 卷积经常用在信号处理中，用于计算信号的延迟累积。
- ▶ 假设一个信号发生器每个时刻 t 产生一个信号 x_t ，其信息的衰减率为 w_k ，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍
 - ▶ 假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$
- ▶ 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加。

卷积

- ▶ 卷积经常用在信号处理中，用于计算信号的延迟累积。
- ▶ 假设一个信号发生器每个时刻 t 产生一个信号 x_t ，其信息的衰减率为 w_k ，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍
 - ▶ 假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$
- ▶ 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加

$$\begin{aligned}y_t &= 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2} \\&= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\&= \sum_{k=1}^3 w_k \cdot x_{t-k+1}.\end{aligned}$$

滤波器 (filter) 或卷积核 (convolution kernel)

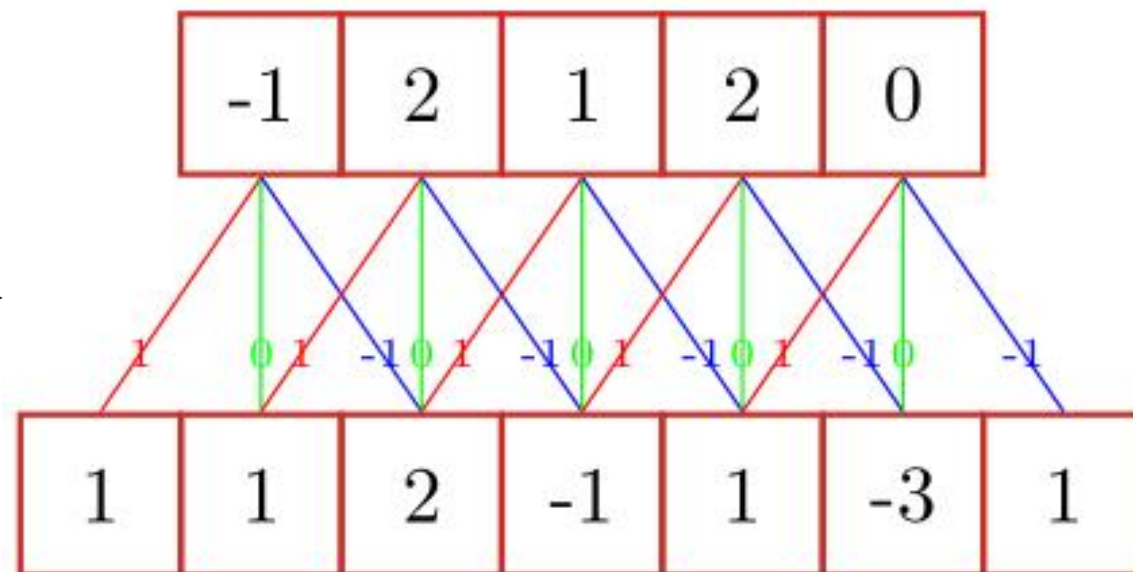
卷积

► 给定一个输入信号序列 x 和滤波器 w ,卷积的输出为:

$$y_t = \sum_{k=1}^K w_k x_{t-k+1}$$

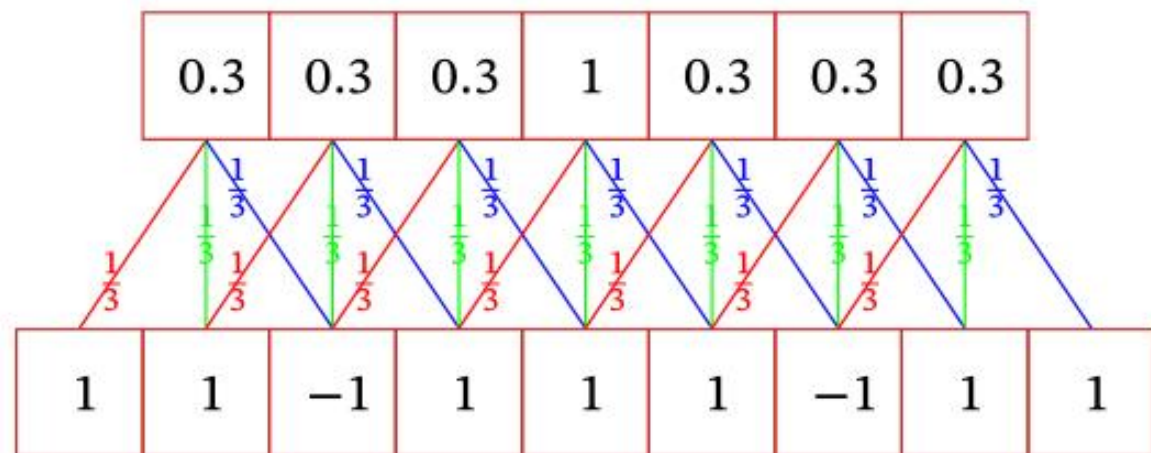
Filter: $[-1, 0, 1]$

翻转对齐



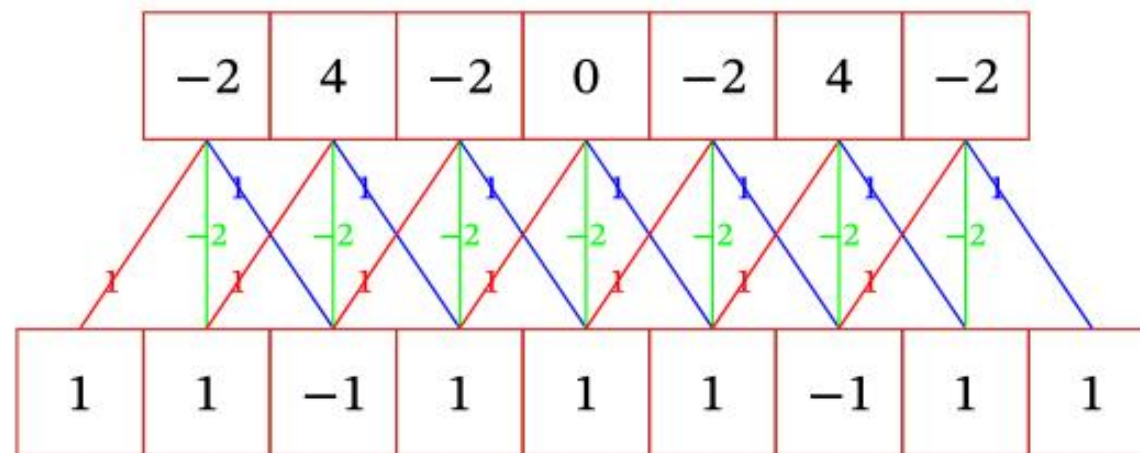
卷积

- 不同的滤波器来提取信号序列中的不同特征



(a) 滤波器 $[1/3, 1/3, 1/3]$

低频信息

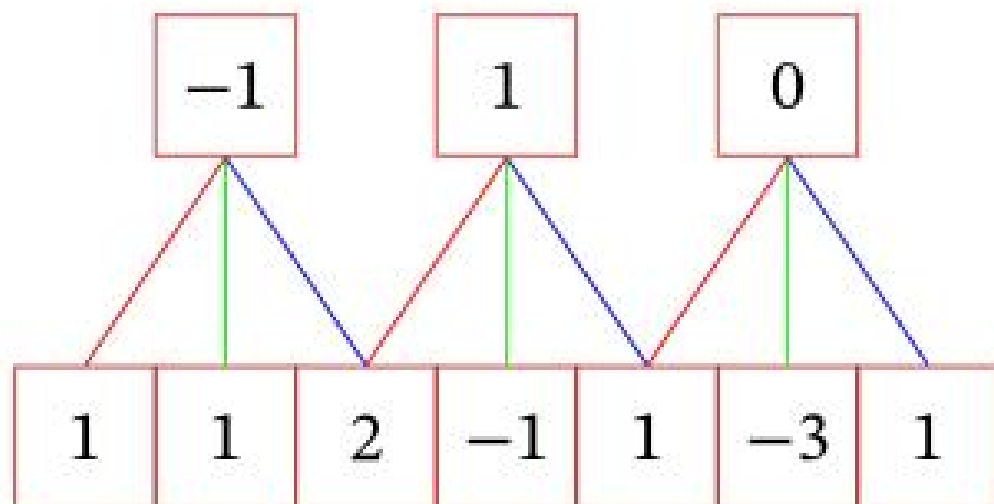


(b) 滤波器 $[1, -2, 1]$

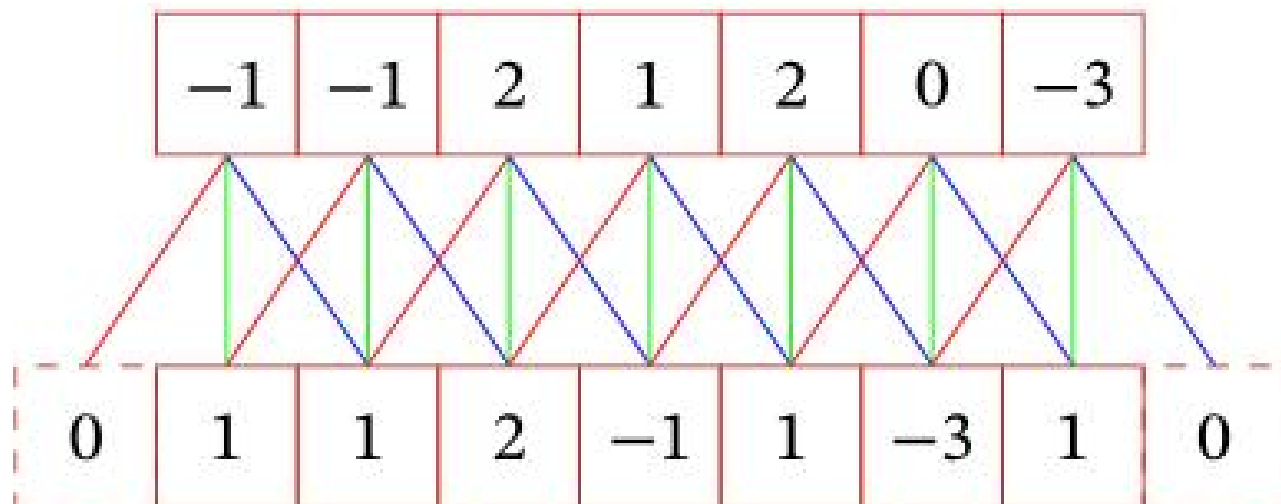
高频信息

卷积扩展

► 引入滤波器的滑动步长 S 和零填充 P



(a) 步长 $S = 2$



(b) 零填充 $P = 1$

卷积类型

► 卷积的结果按输出长度不同可以分为三类：

- **窄卷积**：步长 $T = 1$ ，两端不补零 $P = 0$ ，卷积后输出长度为 $M - K + 1$
- **宽卷积**：步长 $T = 1$ ，两端补零 $P = K - 1$ ，卷积后输出长度 $M + K - 1$
- **等宽卷积**：步长 $T = 1$ ，两端补零 $P = (K - 1)/2$ ，卷积后输出长度 M

假设卷积层的输入神经元个数为 M ，卷积大小为 K ，步长为 S ，在输入两端各填补 P 个 0 (zero padding)，那么该卷积层的神经元数量为 $(M - K + 2P)/S + 1$ 。

- 在早期的文献中，卷积一般默认为**窄卷积**。
- 而目前的文献中，卷积一般默认为**等宽卷积**。

二维卷积

- 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

一个输入信息 $\mathbf{X}$ 和滤波器 $\mathbf{W}$ 的二维卷积定义为

$$\mathbf{Y} = \mathbf{W} * \mathbf{X},$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}.$$

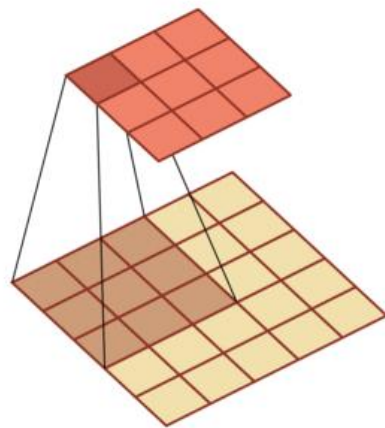
1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

*

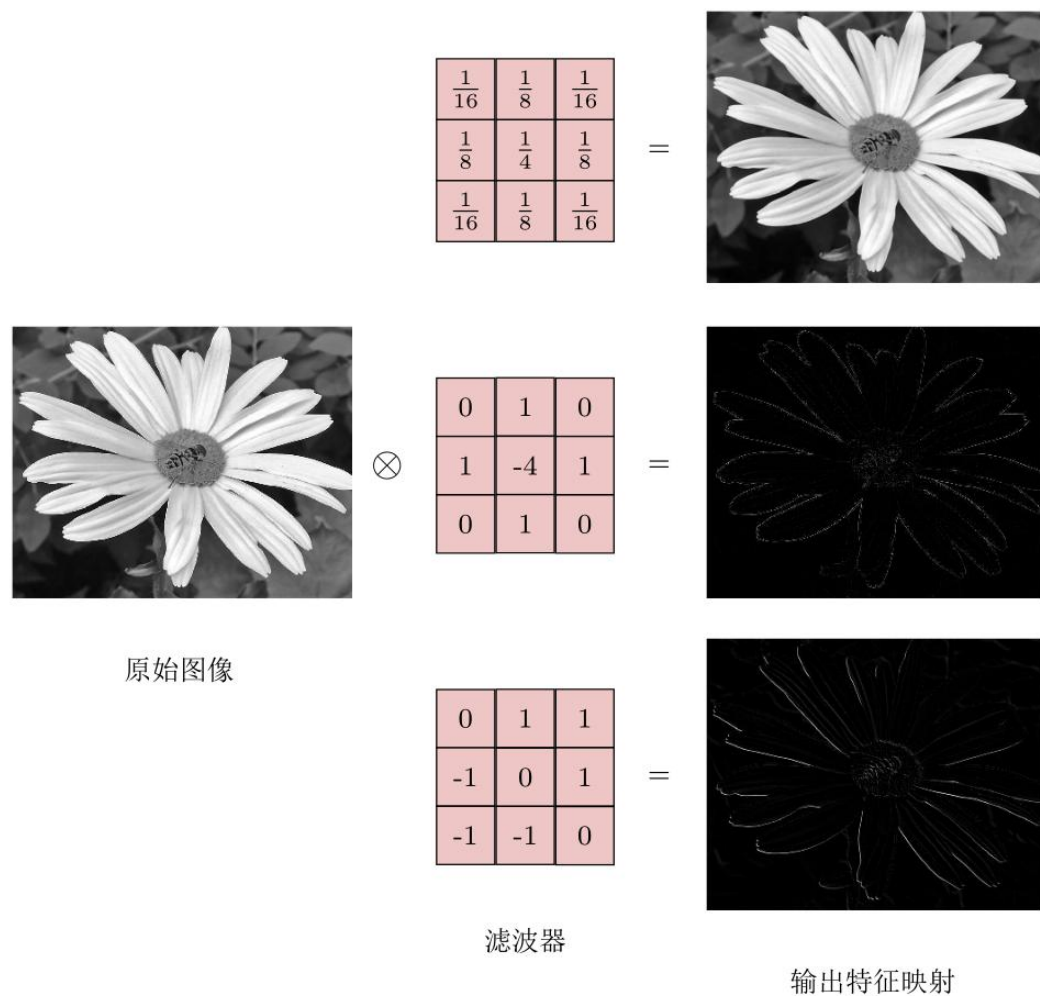
1	0	0
0	0	0
0	0	-1

=

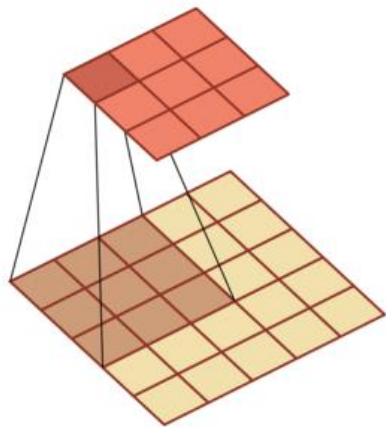
0	-2	-1
2	2	4
-1	0	0



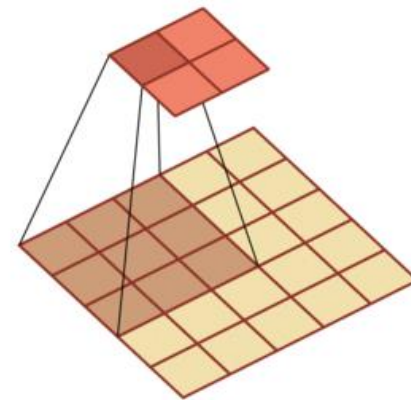
卷积作为特征提取器



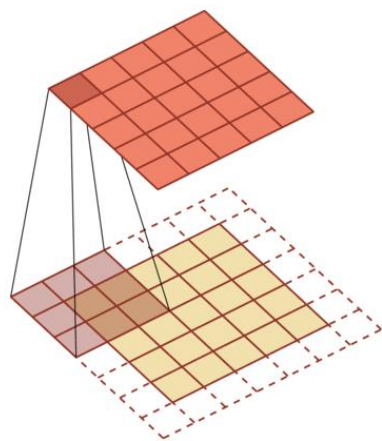
二维卷积



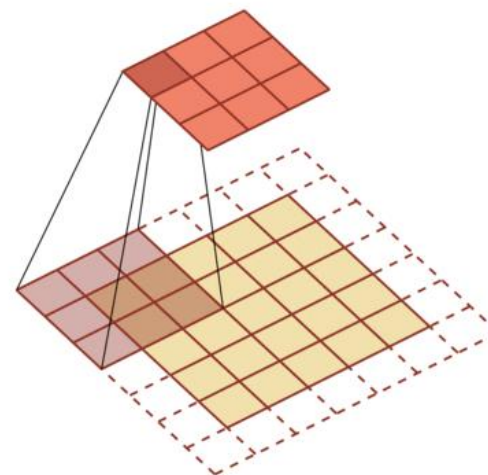
步长1，零填充0



步长2，零填充0



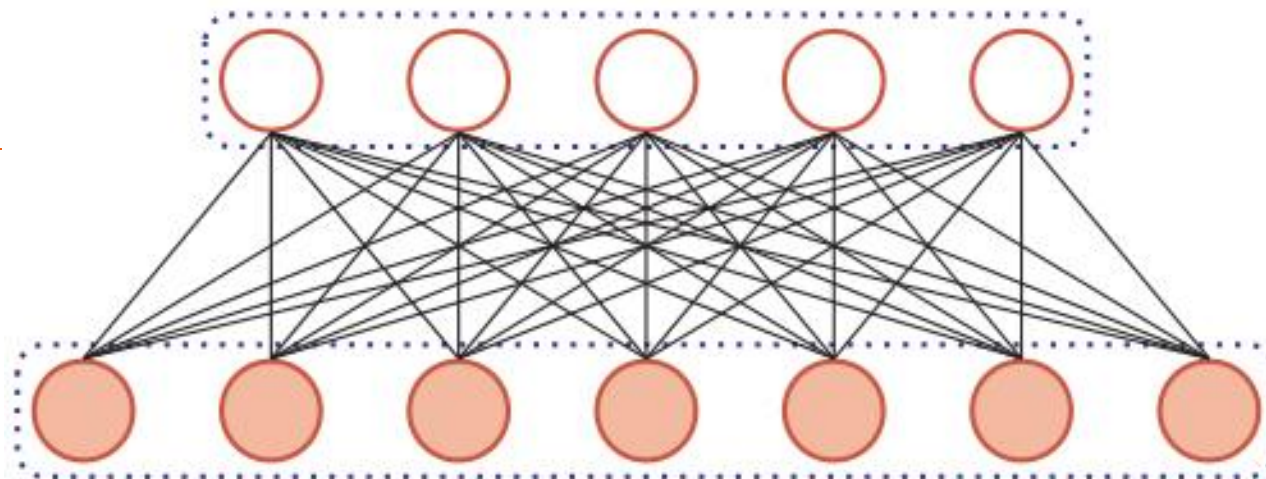
步长1，零填充1



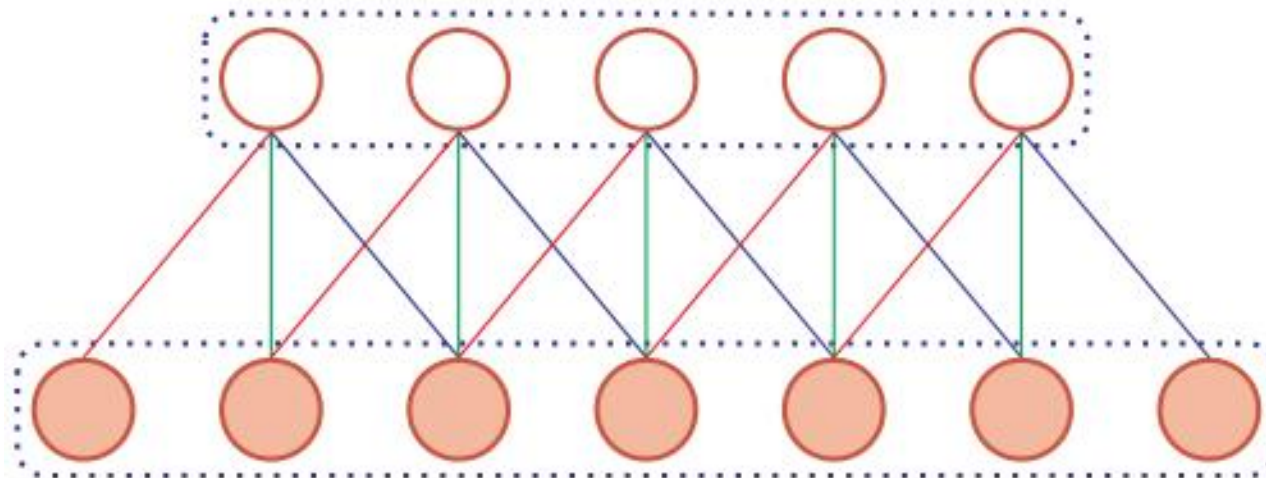
步长2，零填充1

卷积神经网络

► 用卷积层代替全连接层



(a) 全连接层



(b) 卷积层

互相关

- ▶ 计算卷积需要进行卷积核翻转。
- ▶ 卷积操作的目标：提取特征。

翻转是不必要的！

▶ 互相关
$$y_{ij} = \sum_{u=1}^m \sum_{v=1}^n w_{uv} \cdot x_{i+u-1, j+v-1}$$

除非特别声明，
卷积一般指“互相关”。

1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

×-1 ×0 ×0
×0 ×0 ×0
×0 ×0 ×1

*

1	0	0
0	0	0
0	0	-1

=

0	-2	-1
2	2	4
-1	0	0

多个卷积核

▶ 特征映射 (Feature Map) : 图像经过卷积后得到的特征。

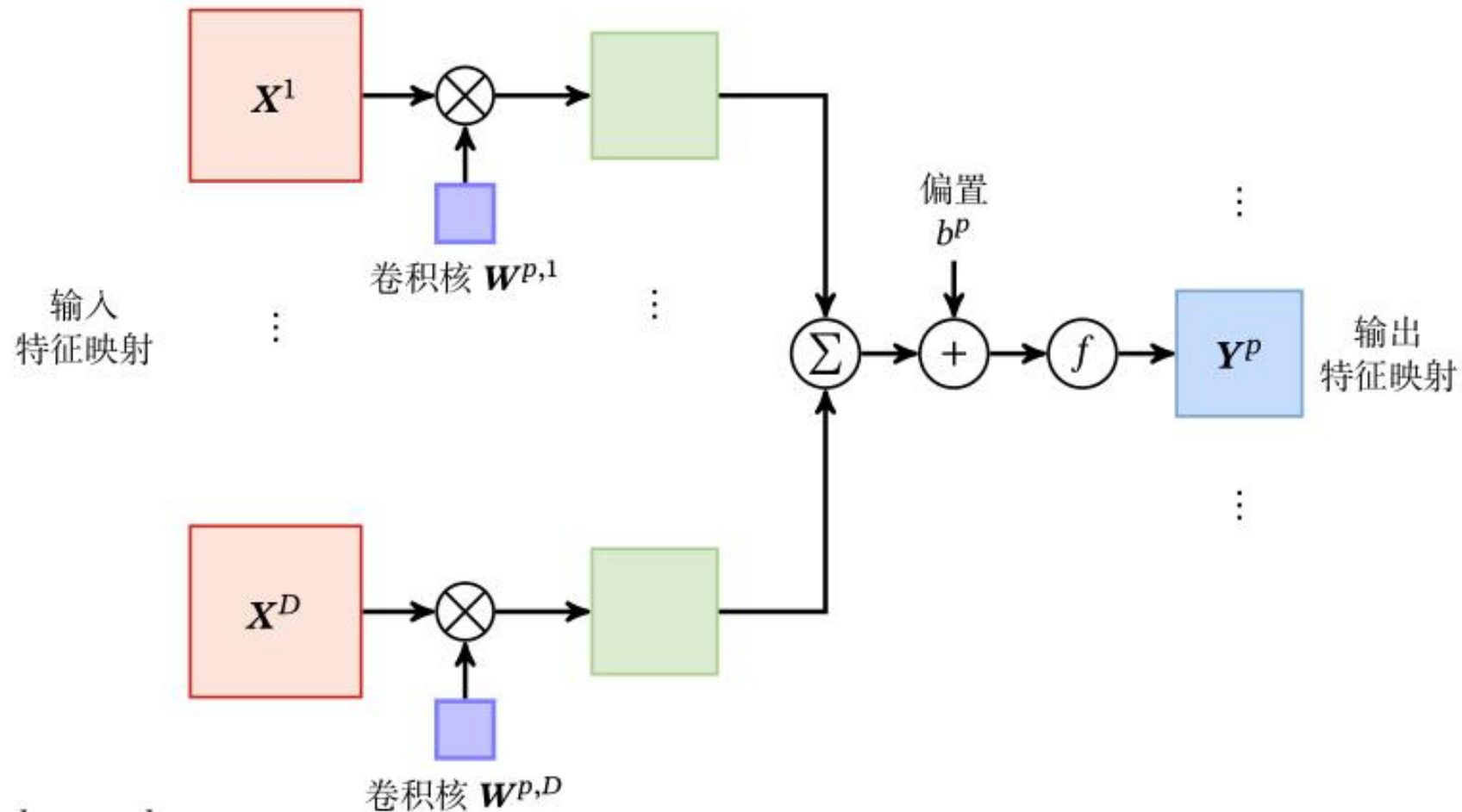
▶ 卷积核看成一个特征提取器

▶ 卷积层

▶ 输入: D 个特征映射 $M \times N \times D$

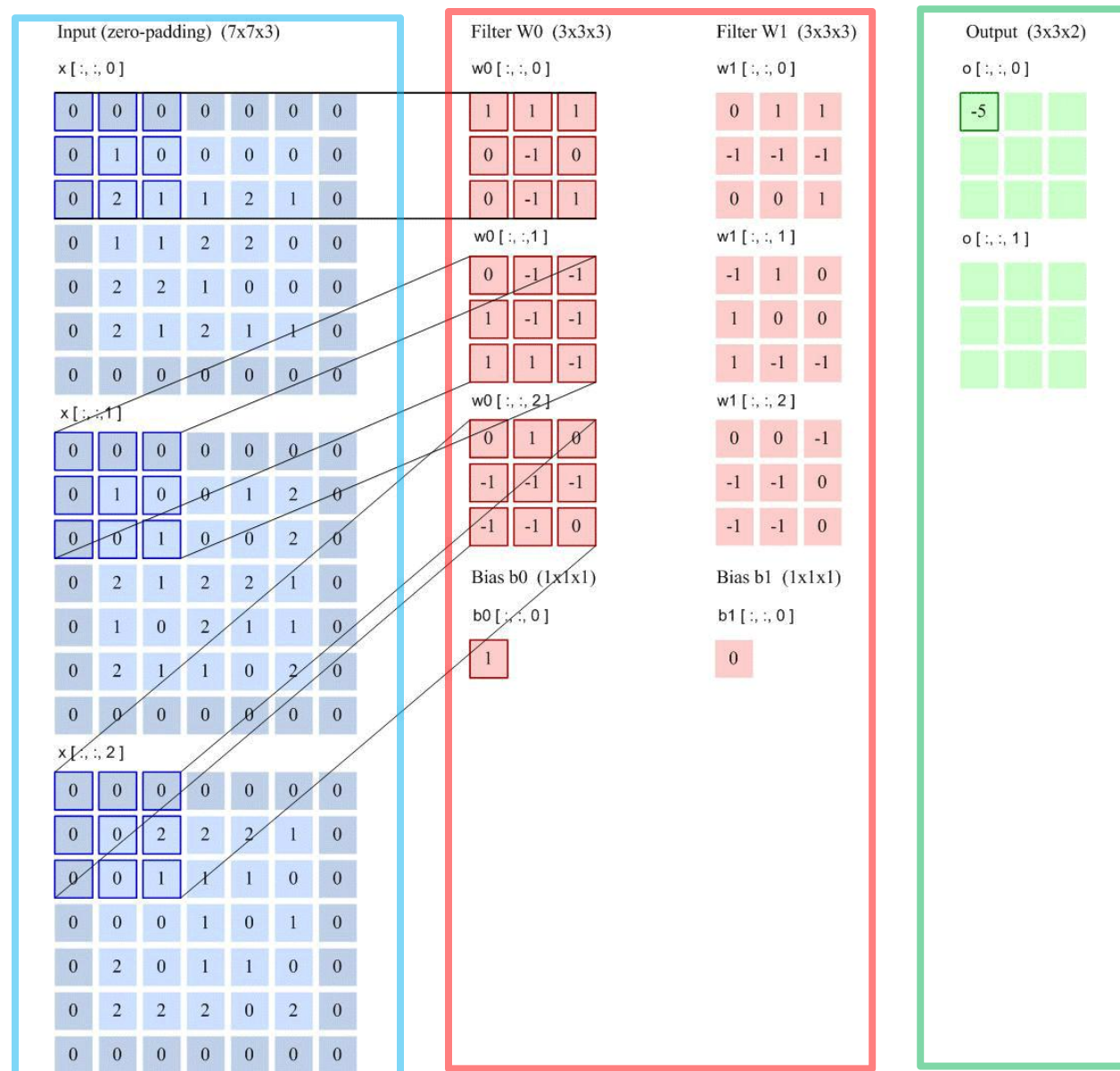
▶ 输出: P 个特征映射 $M' \times N' \times P$

卷积层的映射关系



$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

$$Y^p = f(Z^p).$$



步长2

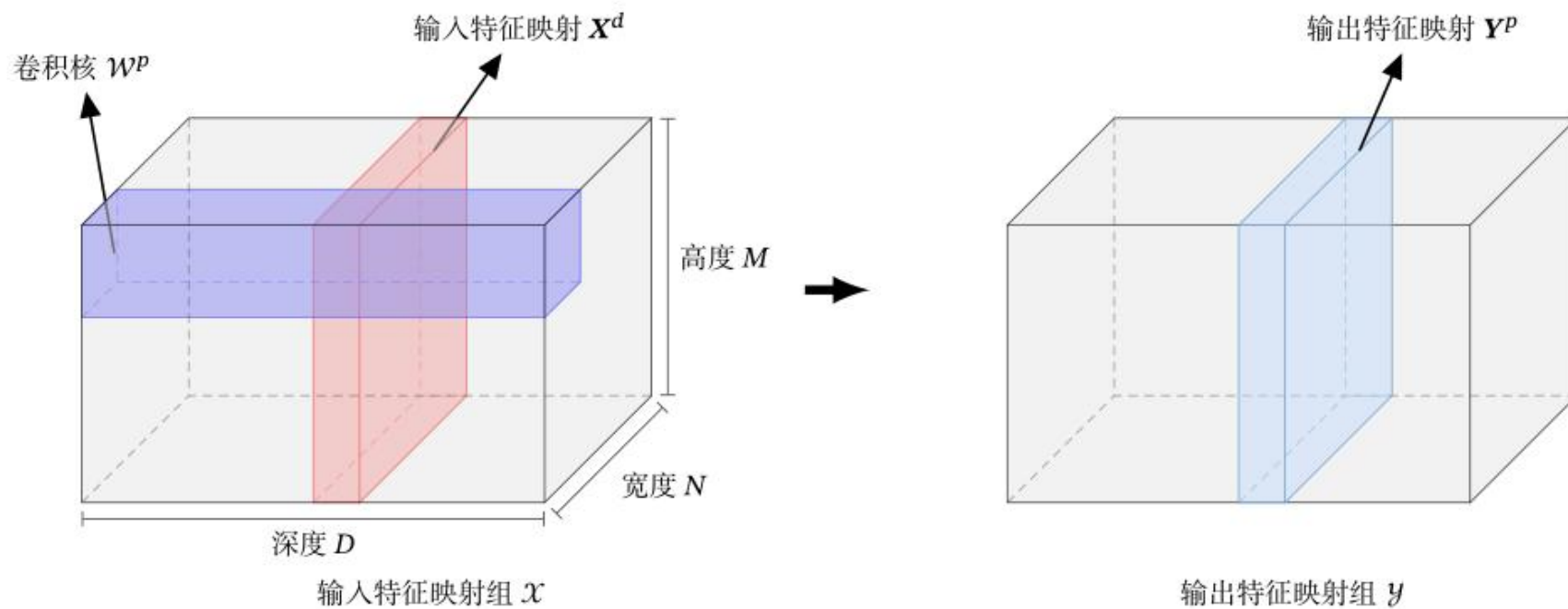
filter (卷积核) 尺寸 3*3*3

filter (卷积核) 个数 2

零填充 1

卷积层

► 典型的卷积层为3维结构

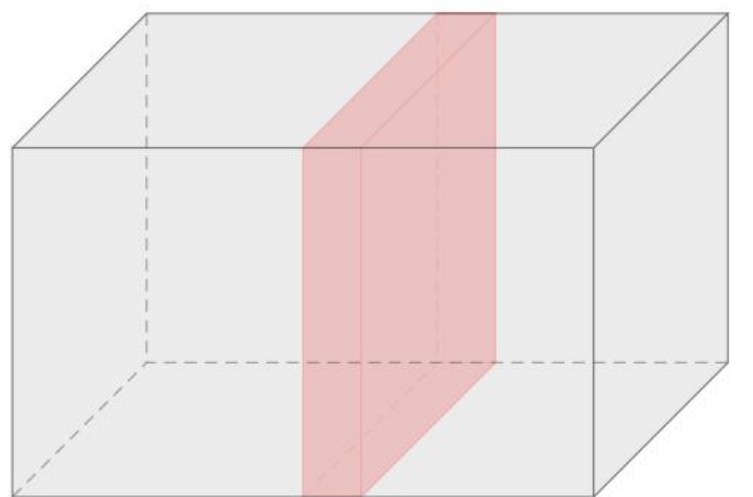


$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

$$Y^p = f(Z^p).$$

汇聚层

- 卷积层虽然可以显著减少连接的个数,但是每一个特征映射的神经元个数并没有显著减少。



输入特征映射组 $\mathcal{X}$

1	1	2	0
0	1	1	2
0	0	1	3
0	0	1	1

输入特征映射 $\mathbf{X}^d$

最大汇聚



输出特征映射组 $\mathcal{Y}$

1	2
0	3

输出特征映射 $\mathbf{Y}^d$

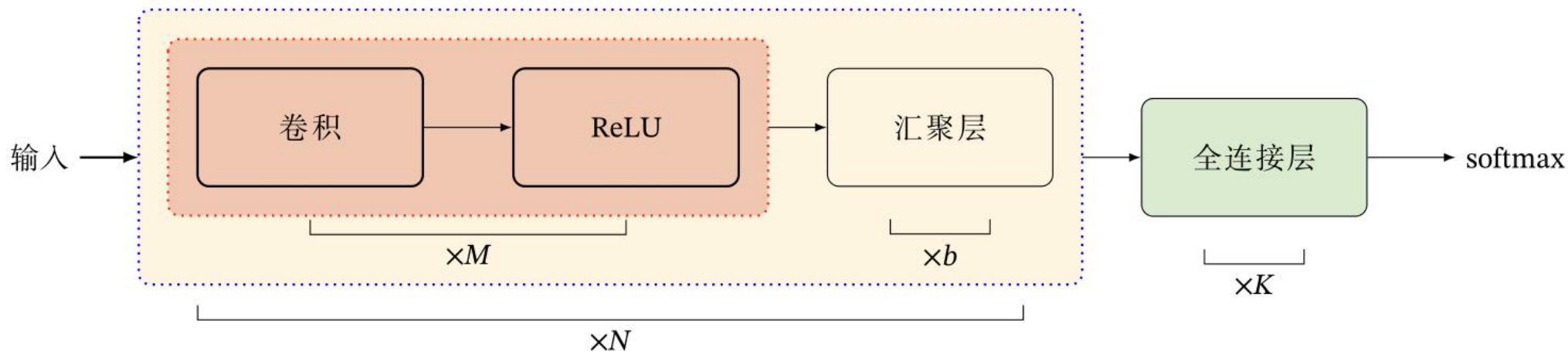
卷积网络结构

► 卷积网络是由卷积层、汇聚层、全连接层交叉堆叠而成。

► 趋向于小卷积、大深度

► 趋向于全卷积

► 典型
结构



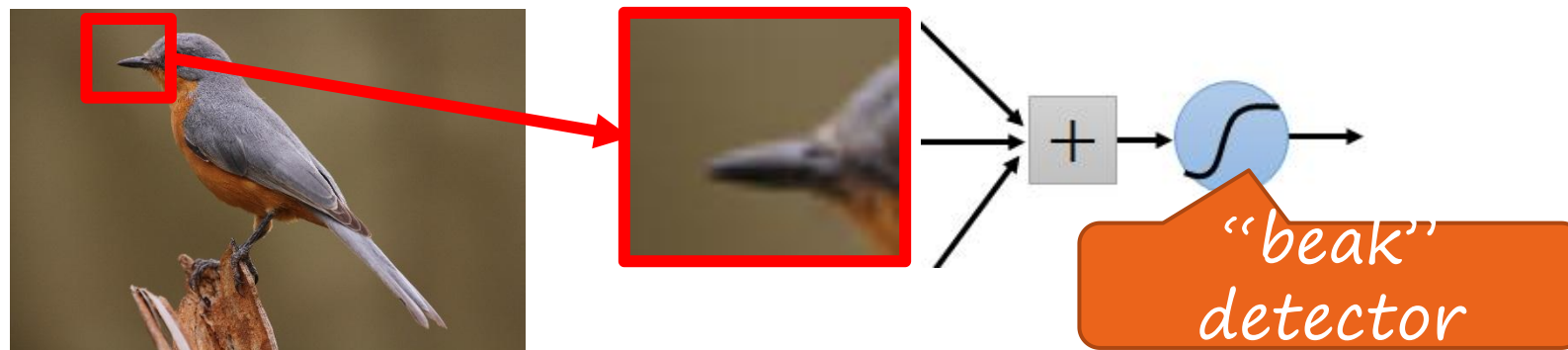
► 一个卷积块为连续 M 个卷积层和 b 个汇聚层（ M 通常设置为2~5， b 为0或1）。一个卷积网络中可以堆叠 N 个连续的卷积块，然后在接着 K 个全连接层（ N 的取值区间比较大，比如1~100或者更大； K 一般为0~2）。

Why CNN for Image

- *Some patterns are much smaller than the whole image*

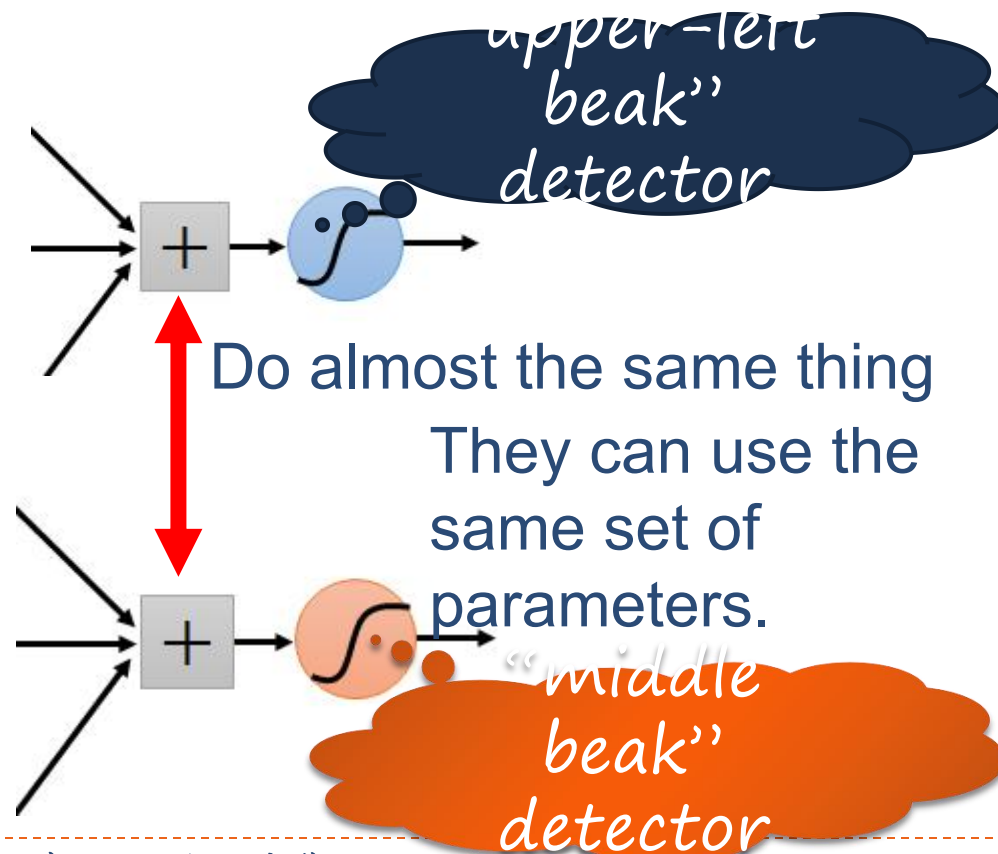
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for Image

- ▶ *The same patterns appear in different regions.*



Why CNN for Image

- *Subsampling the pixels will not change the object*

bird



subsampling

bird



We can subsample the pixels to make image

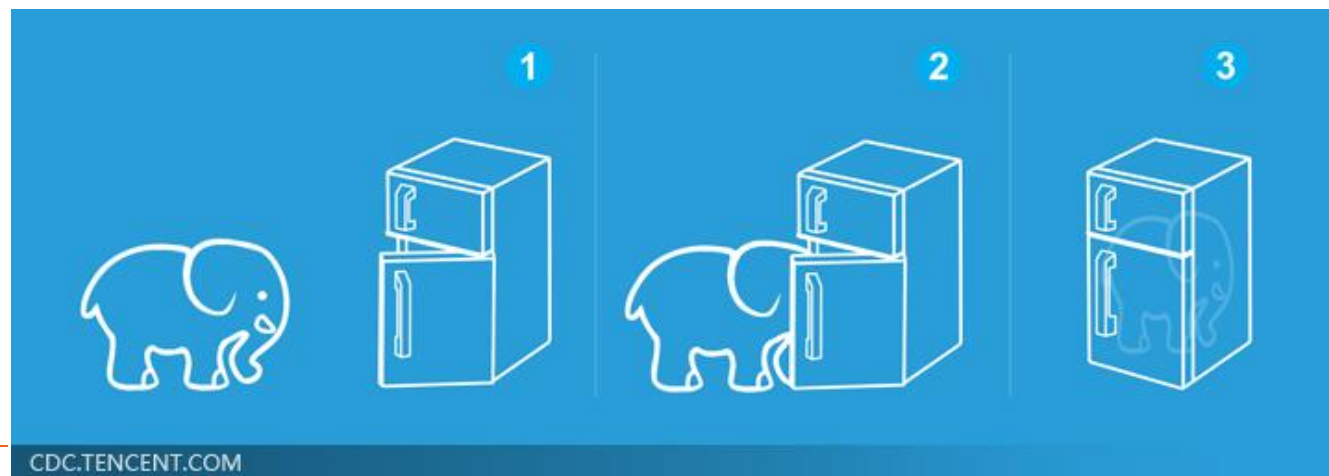


smaller
Less parameters for the network to process the image

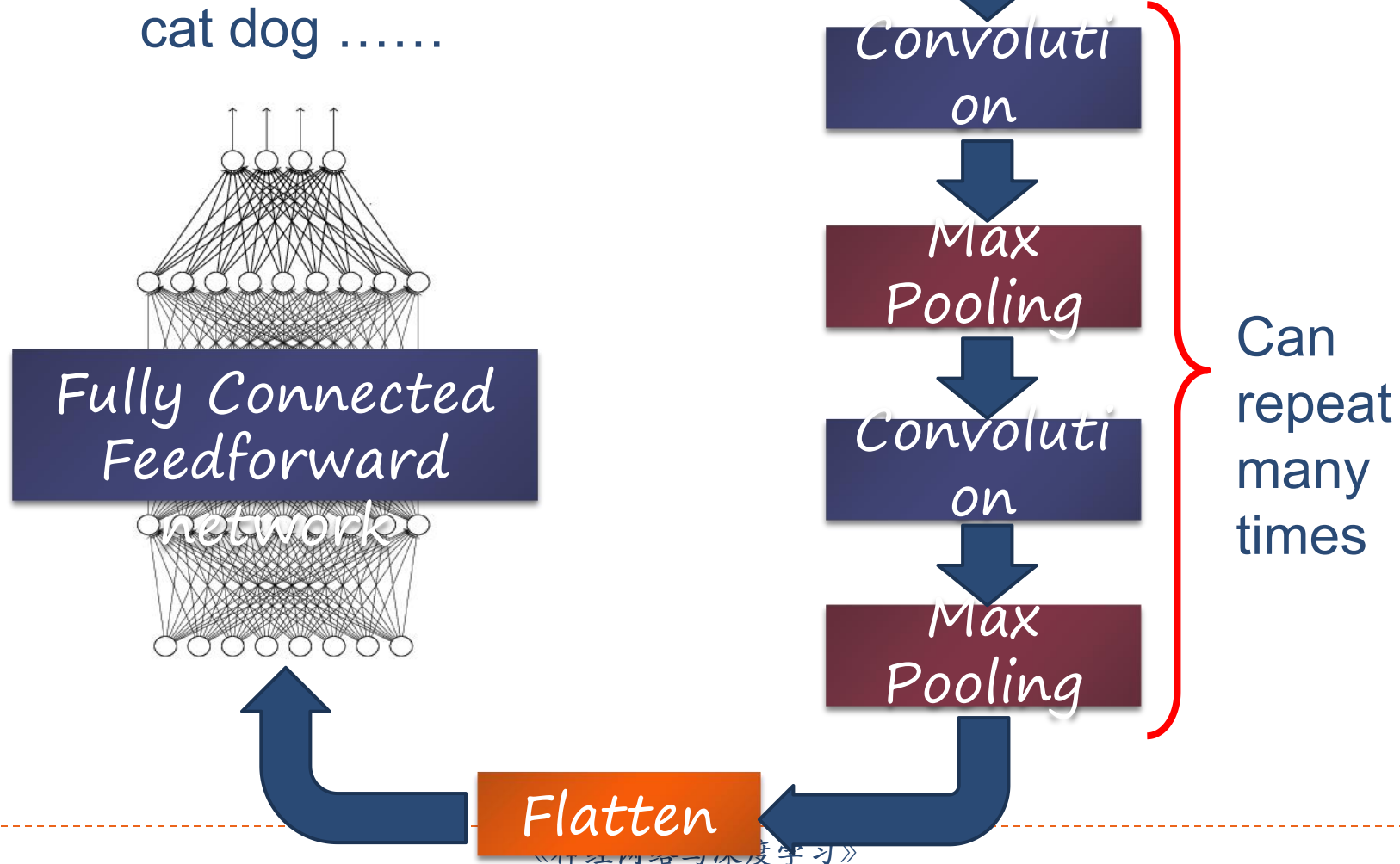
Three Steps for Deep Learning



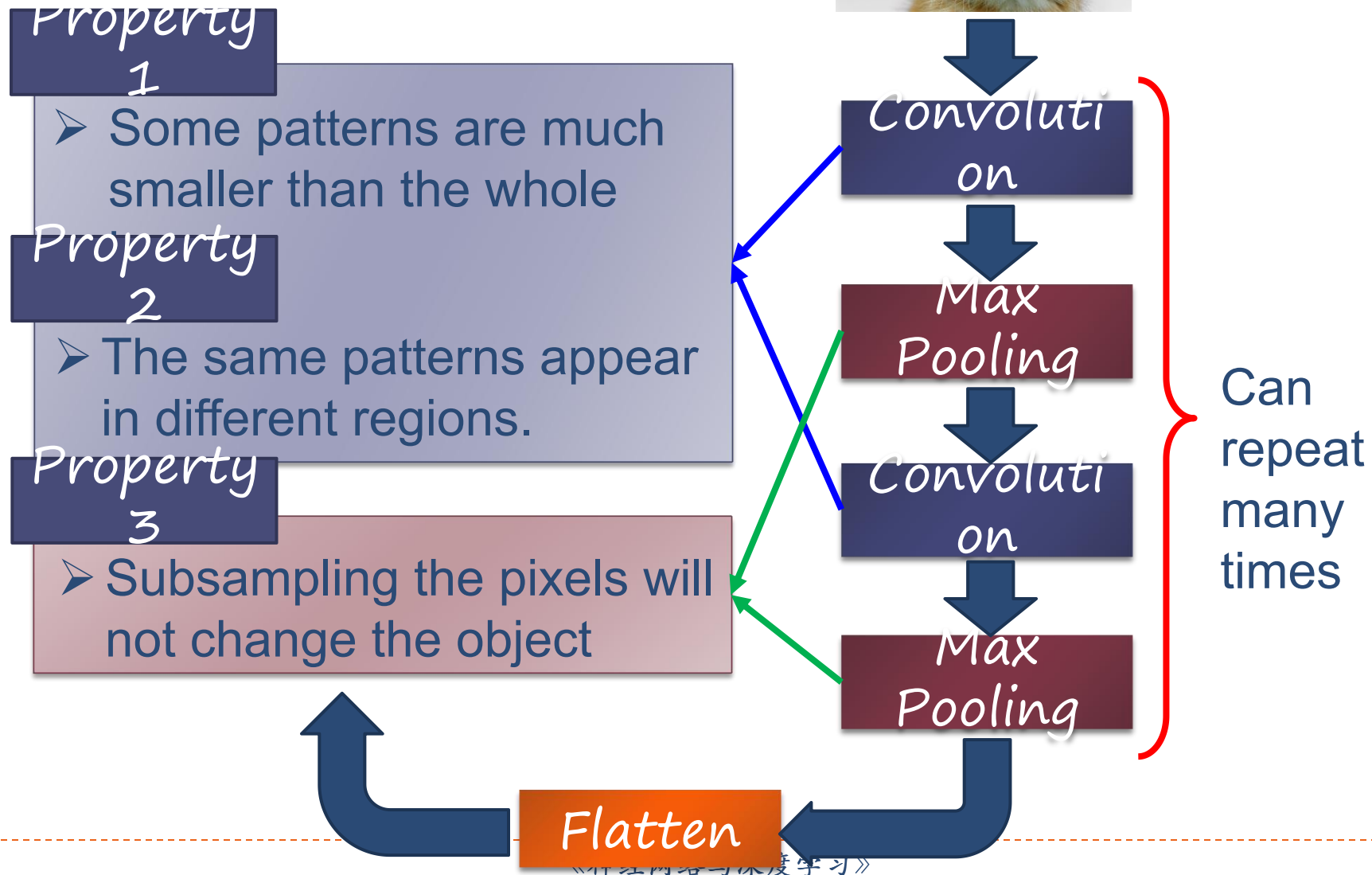
Deep Learning is so simple



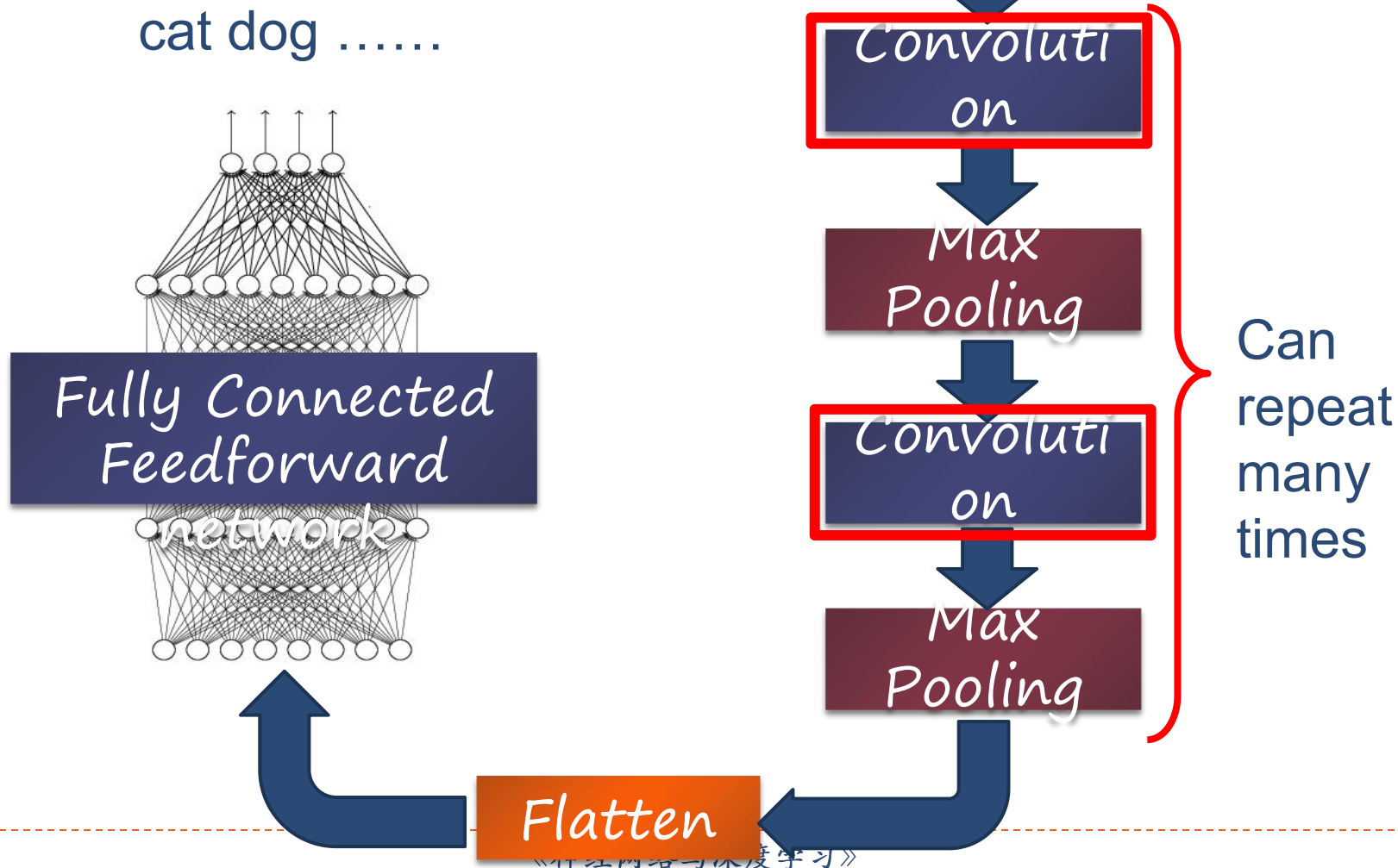
The whole CNN



The whole CNN



The whole CNN



CNN – Convolution

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Those are the network parameters to be learned.

1	-	-
	1	1
-	1	-
1		1
-	1	-
1		1
-	1	-
1		1
-	1	-
1		1

Filter 1

Matrix

Filter 2

Matrix

Property 1

Each filter detects a small pattern (3 x 3).

CNN – Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-	-
-	1	-
-	-	1
1	1	

Filter 1

3
-
1

CNN – Convolution

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-	-
-	1	1
1	-	1
-	-	1
1	1	-

Filter 1



We set stride=1 below

CNN – Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-	-
-	1	1
-	1	-
1	-	1

Filter 1

3	-	-	-
1	1	3	1
-	1	0	-
3	3	0	3
-	-	0	1
3	3	0	1
3	2	2	1
-	-	-	-

Property
2

CNN – Convolution

stride=1

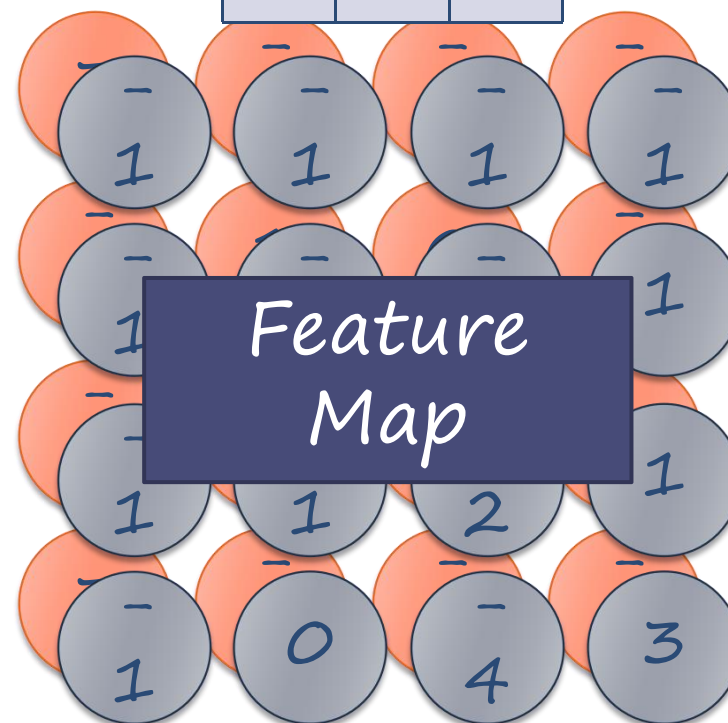
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Do the same process
for every filter

-	1	-
1		1
-	1	-
1		1
-	1	-
1		1

Filter 2



4 x 4 image

CNN – Zero Padding

0	0	0				
0	1	0	0	0	0	1
0	0	1	0	0	1	0
	0	0	1	1	0	0
	1	0	0	0	1	0
	0	1	0	0	1	0
	0	0	1	0	1	0
						0
						0
						0

6 x 6 image

1	-	-
	1	1
-	1	-
1		1
-	-	1
1	1	

Filter 1

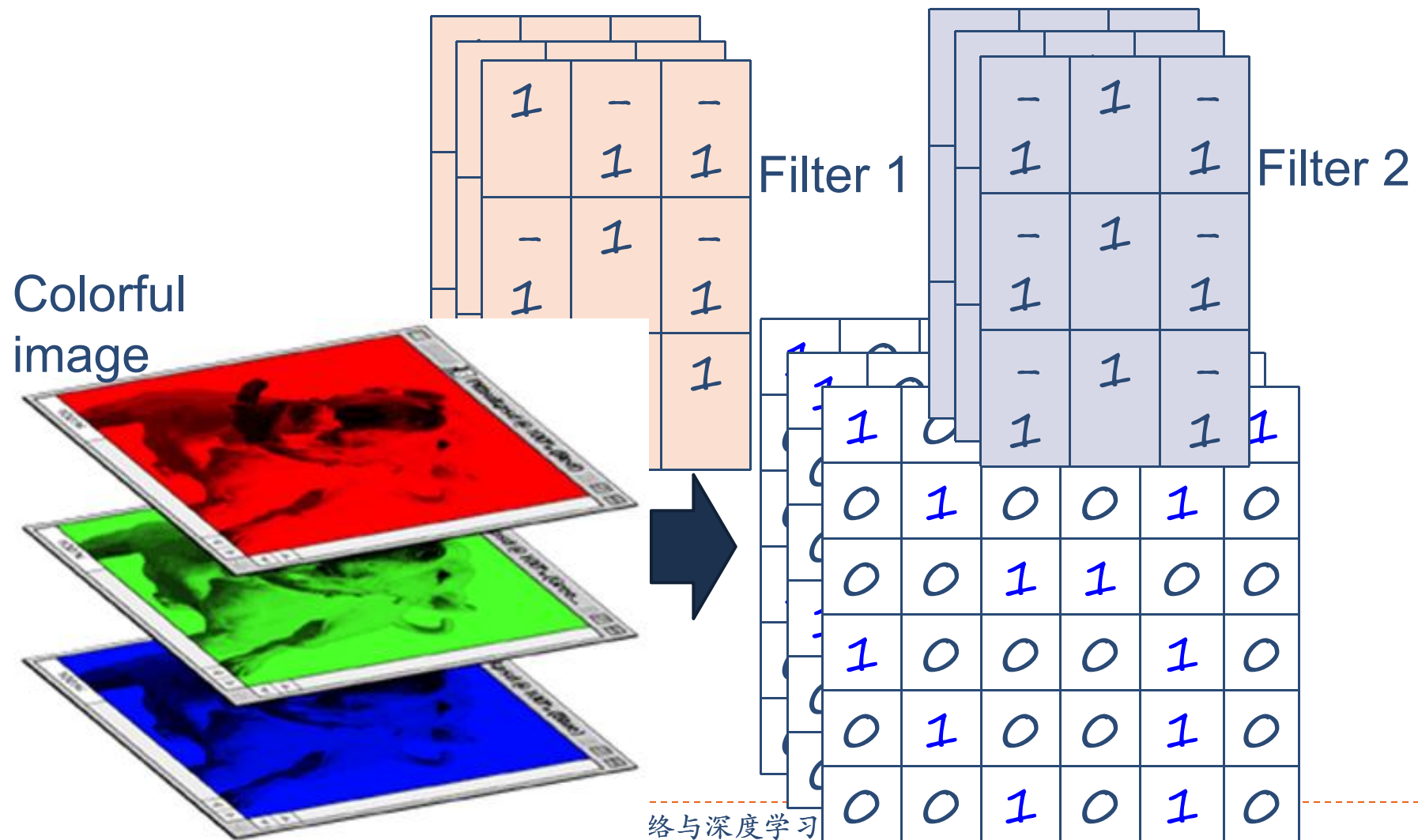
You will get another 6 x 6 images in this way



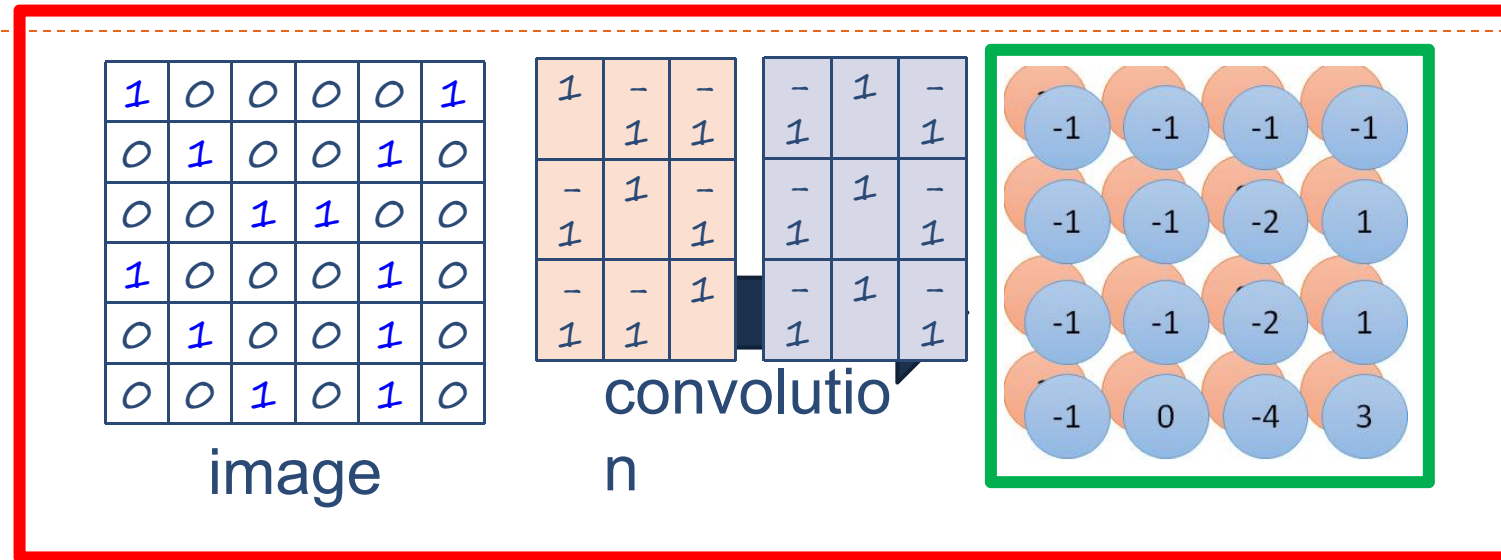
Zero padding

$$(M - K + 2P) / S + 1$$

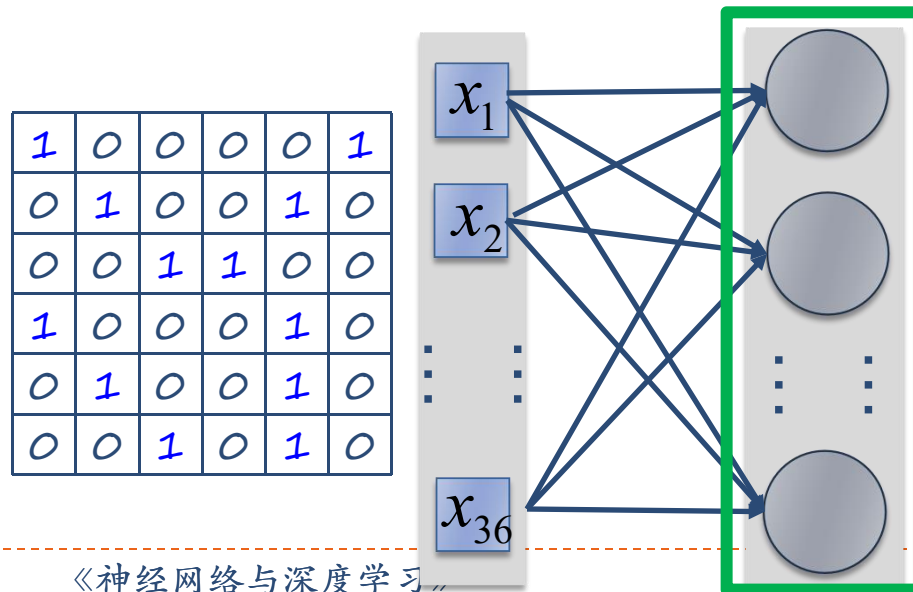
CNN – Colorful image

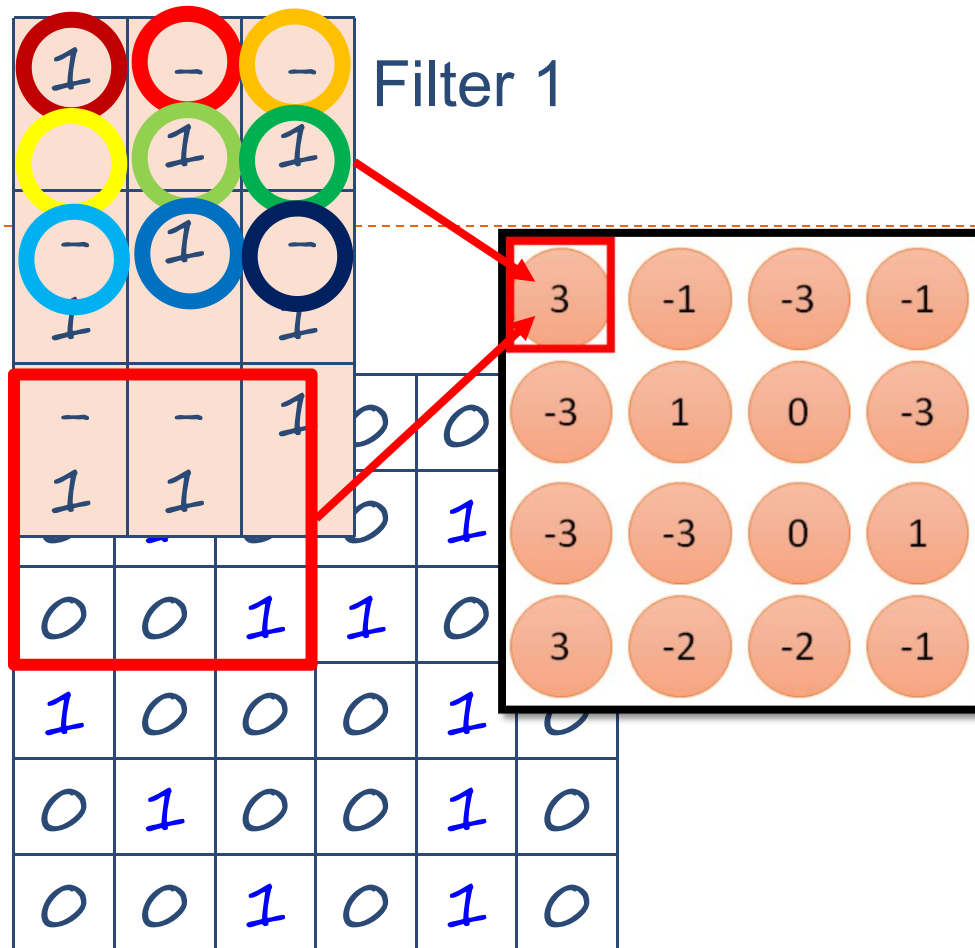


Convolution v.s. Fully Connected



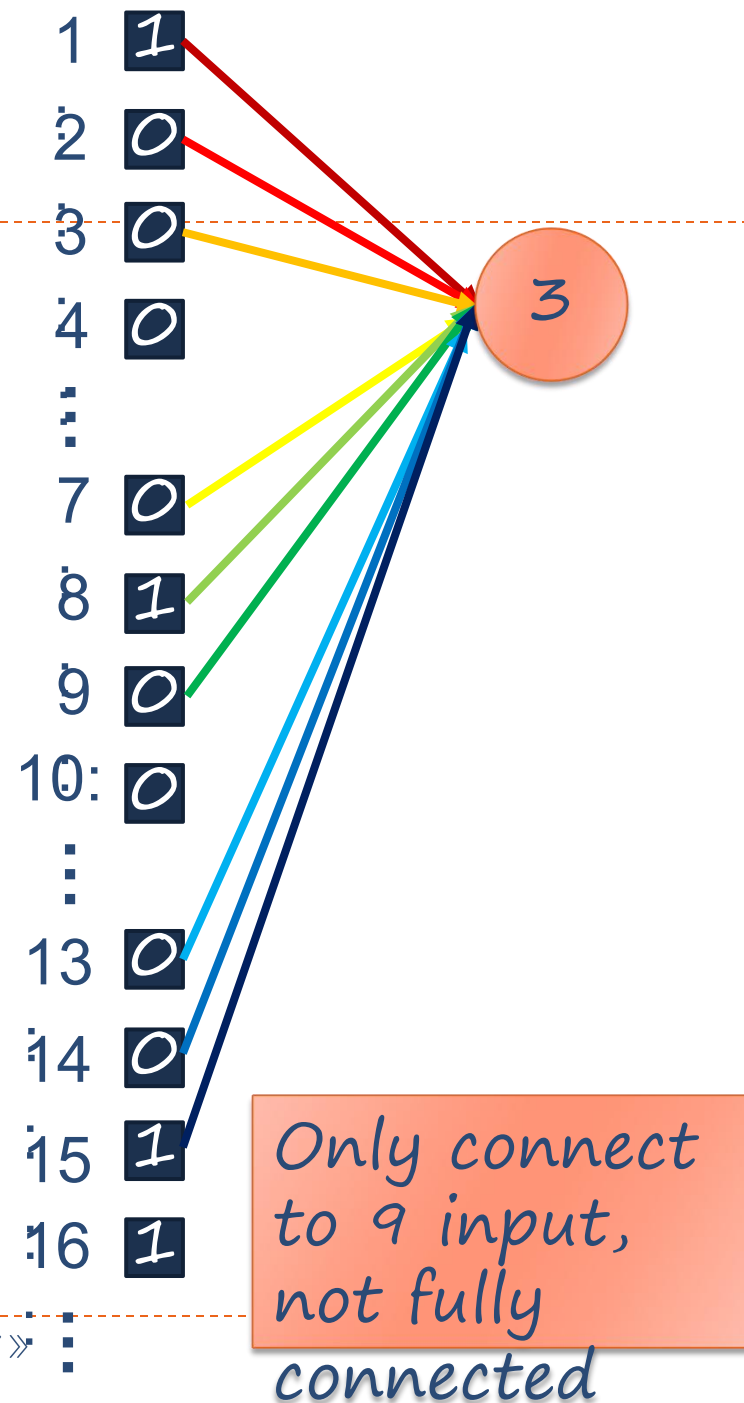
Fully-connected

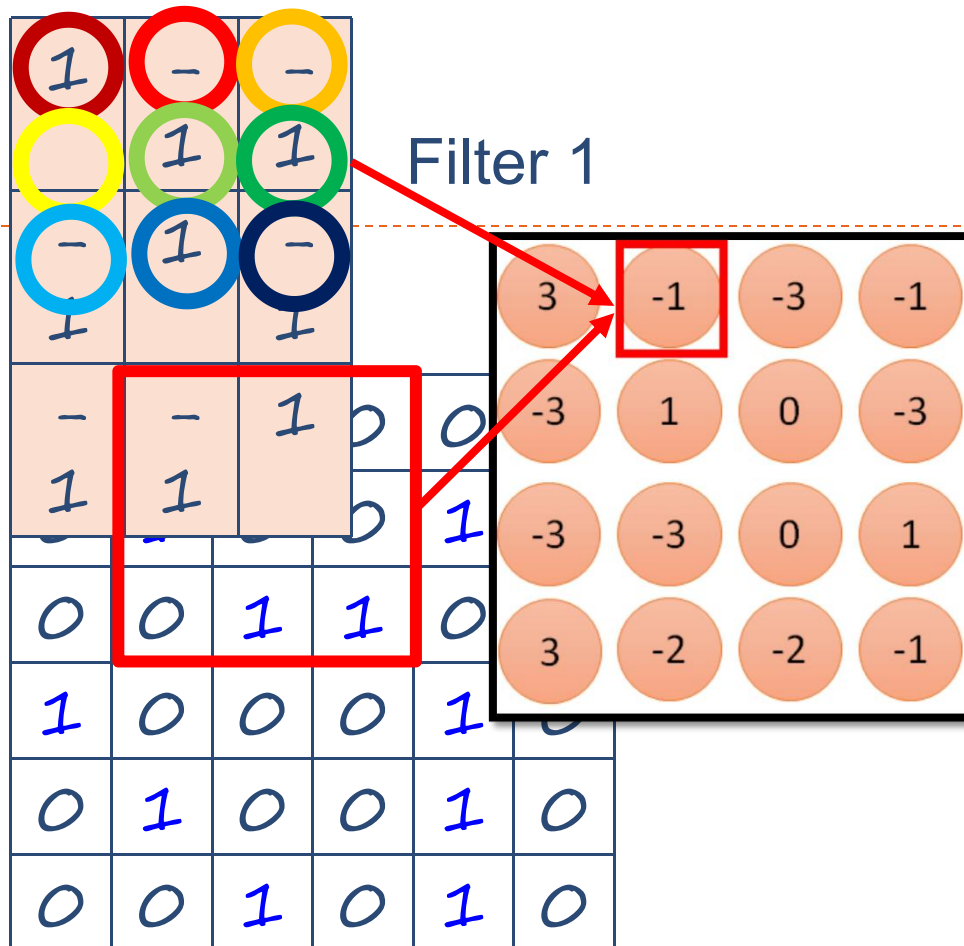




6 x 6 image

Less
parameters!



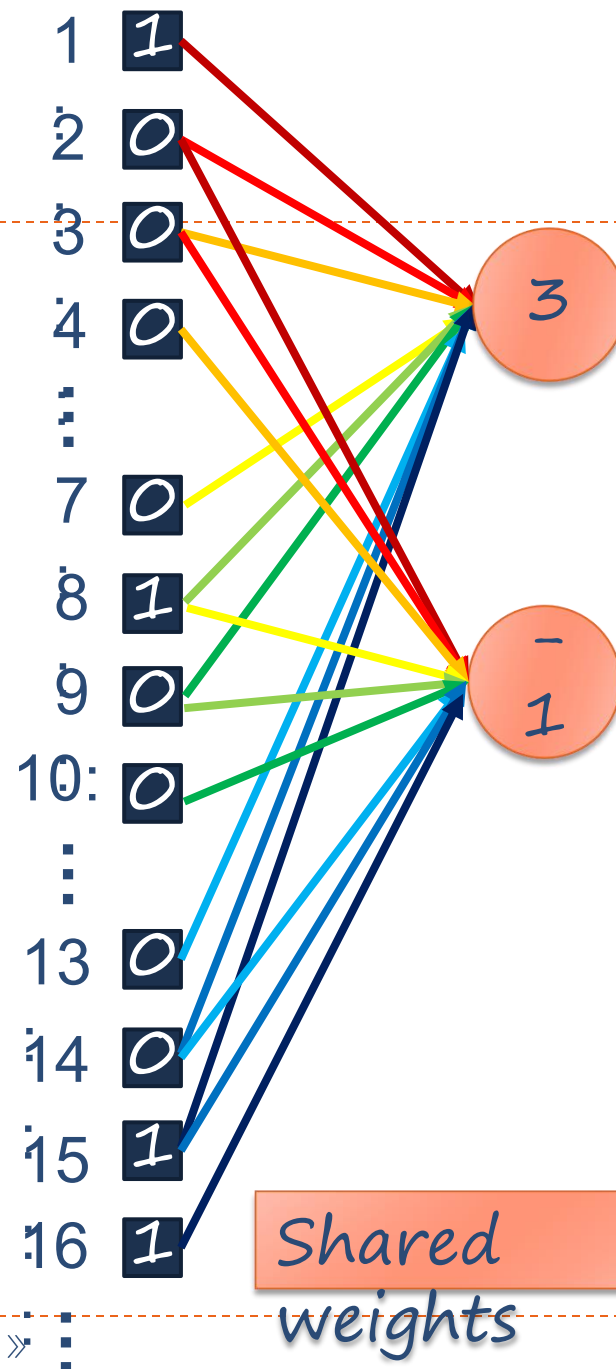


6 x 6 image

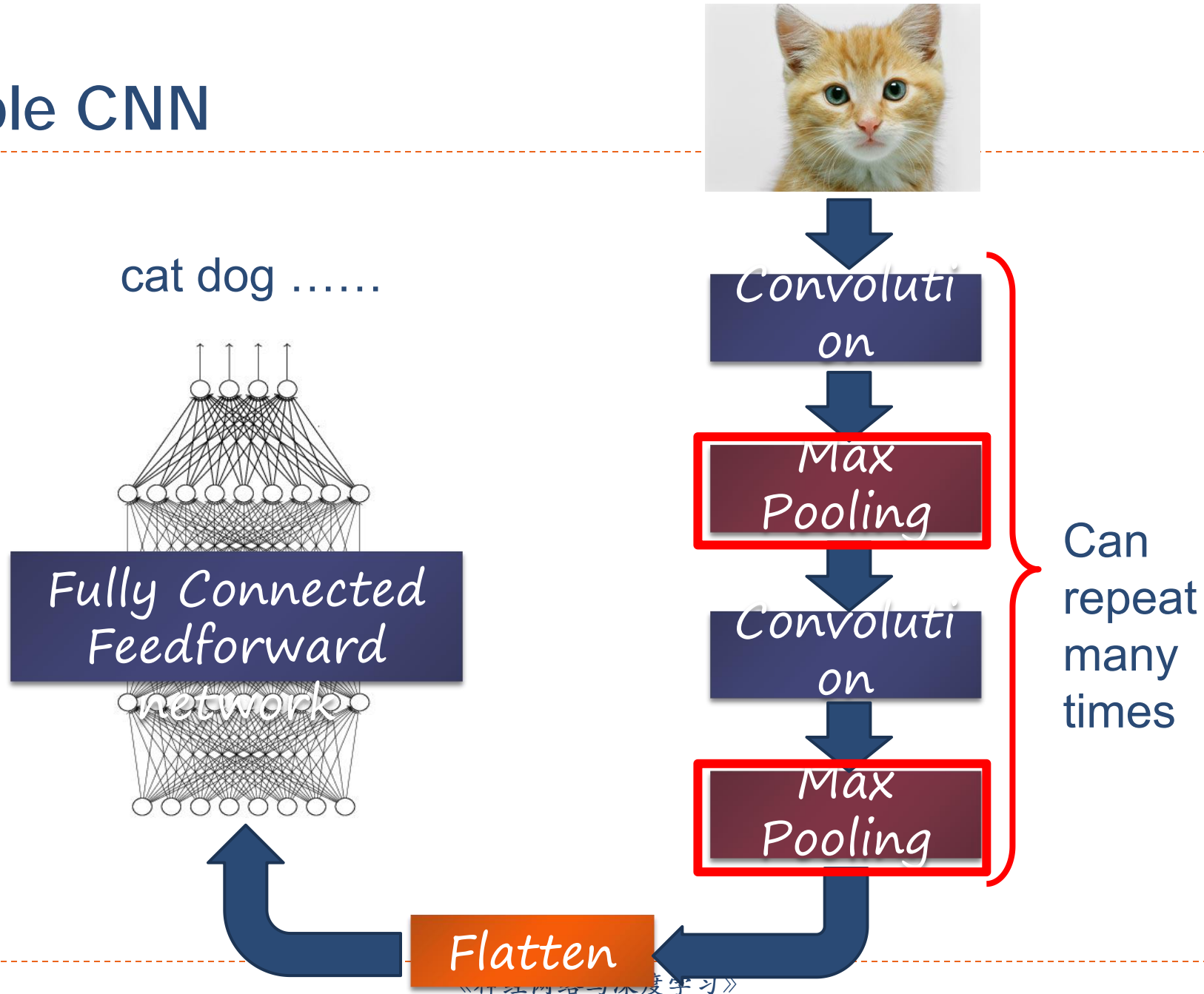
Less

Even less

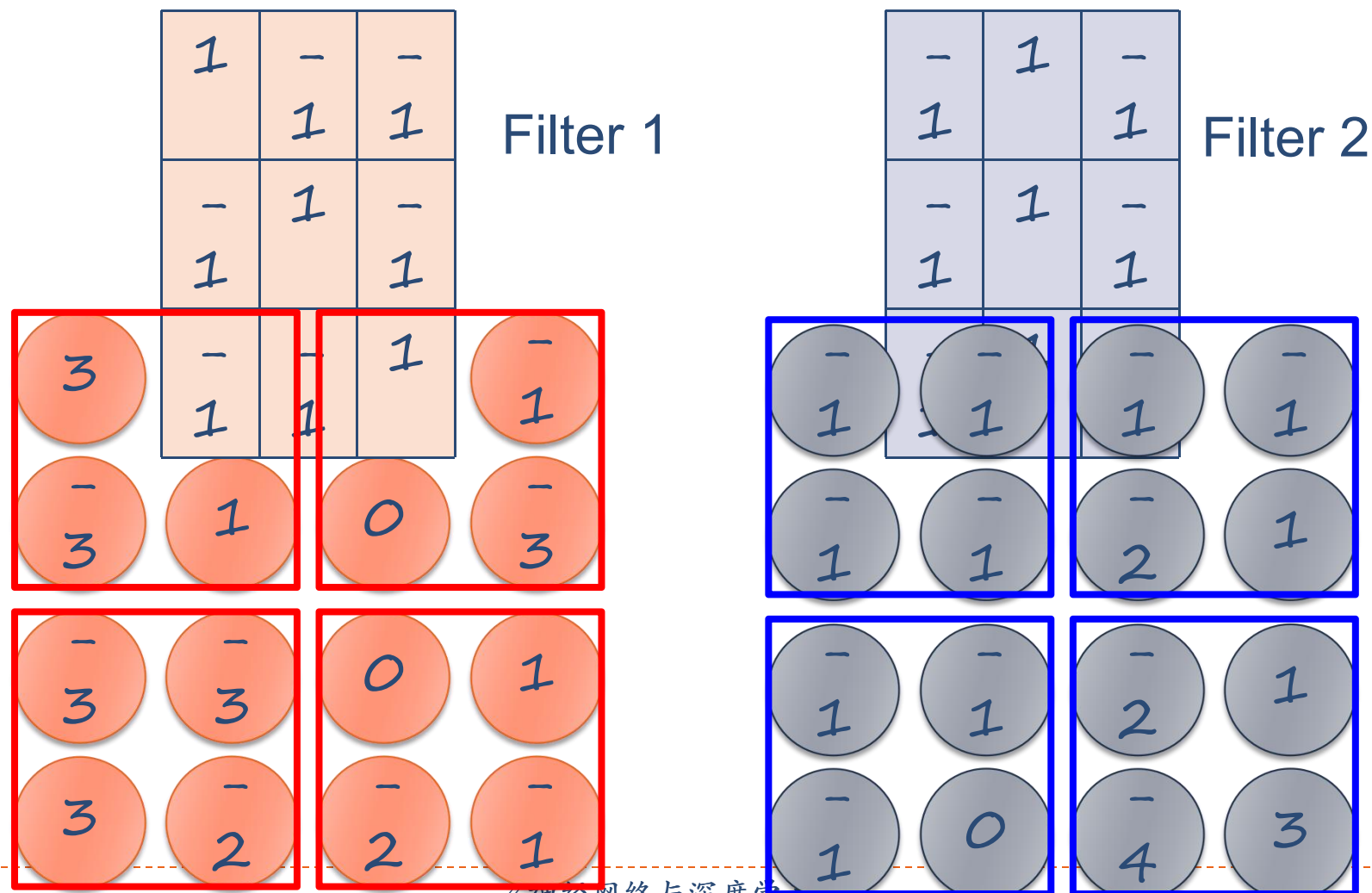
parameters!



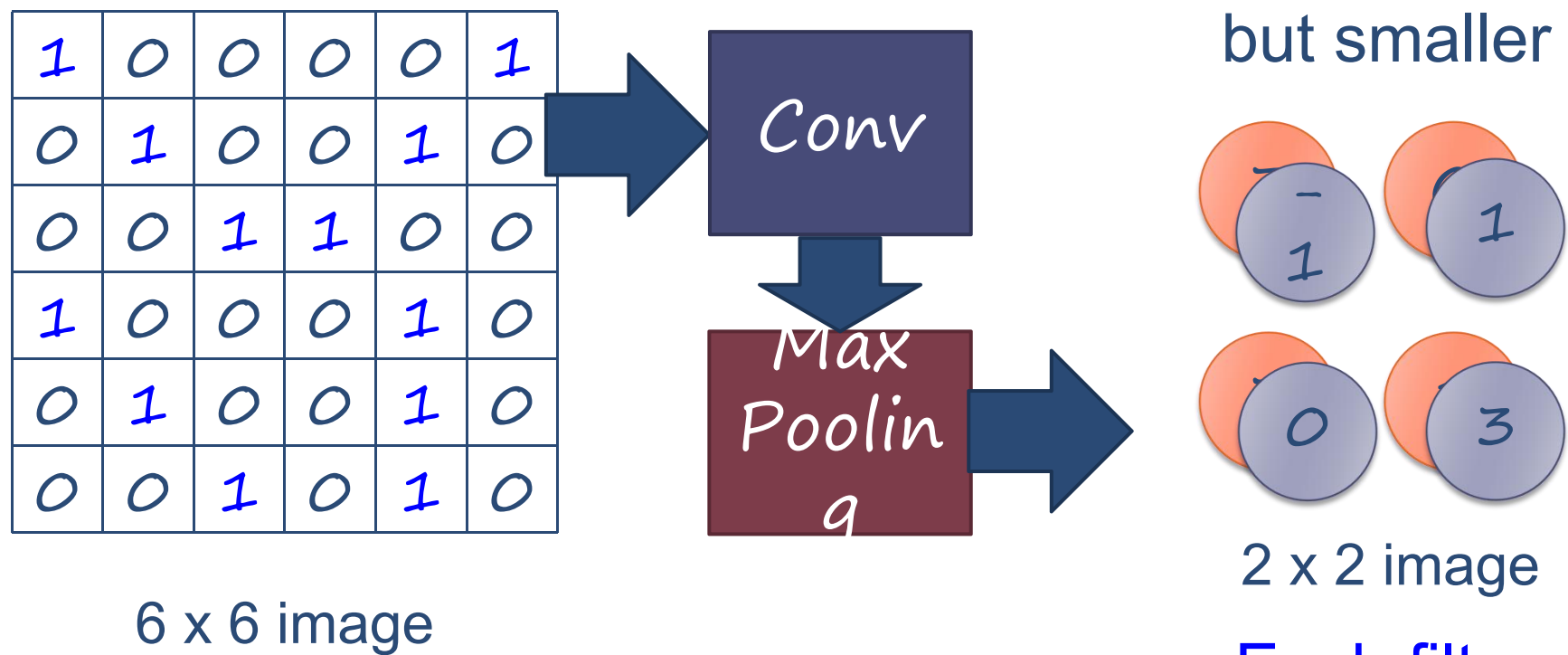
The whole CNN



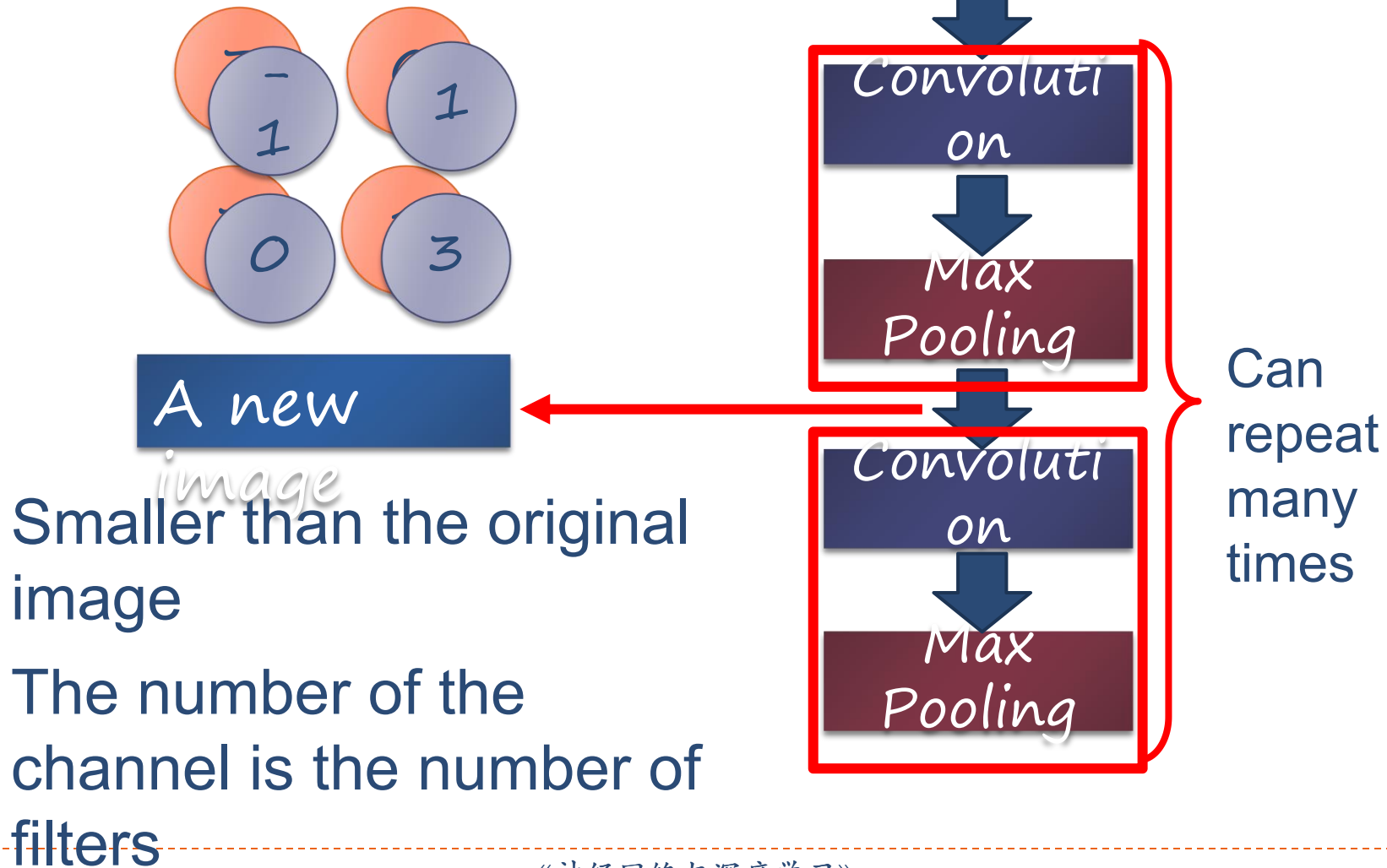
CNN – Max Pooling



CNN – Max Pooling

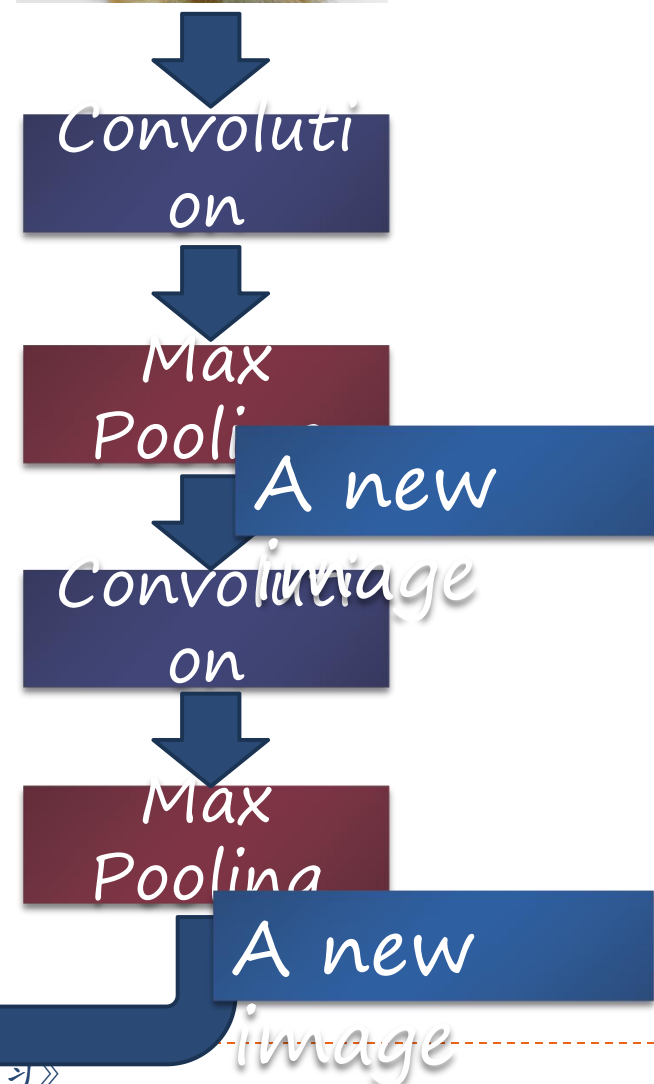
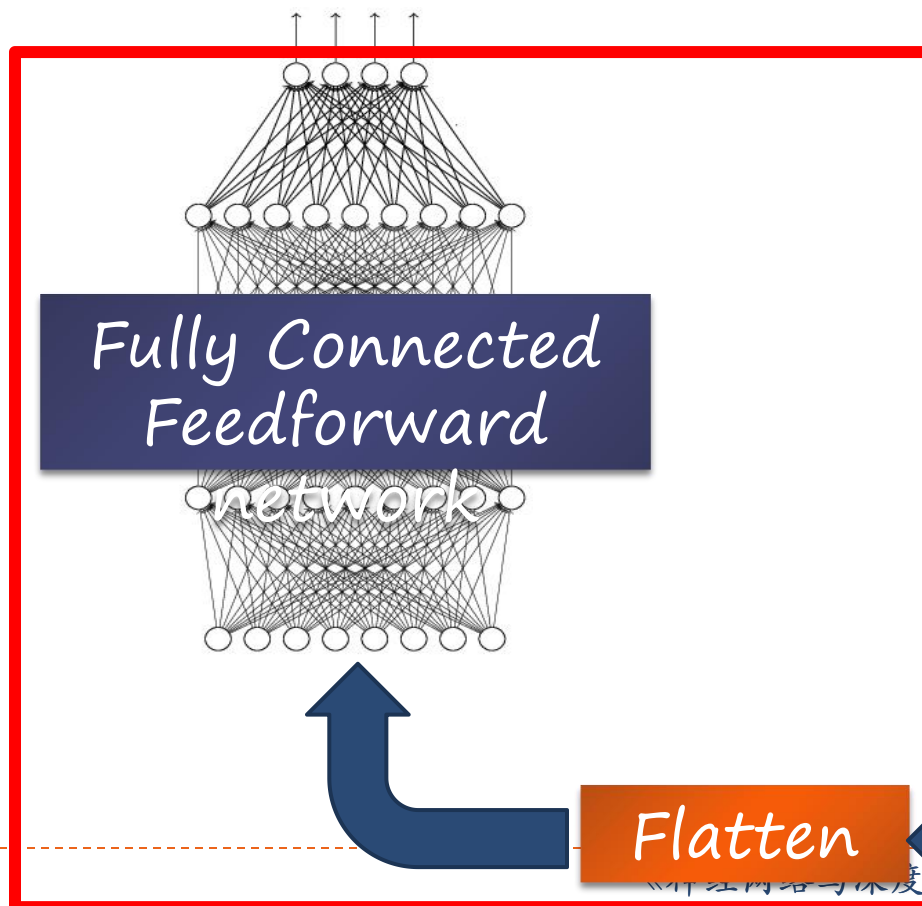


The whole CNN

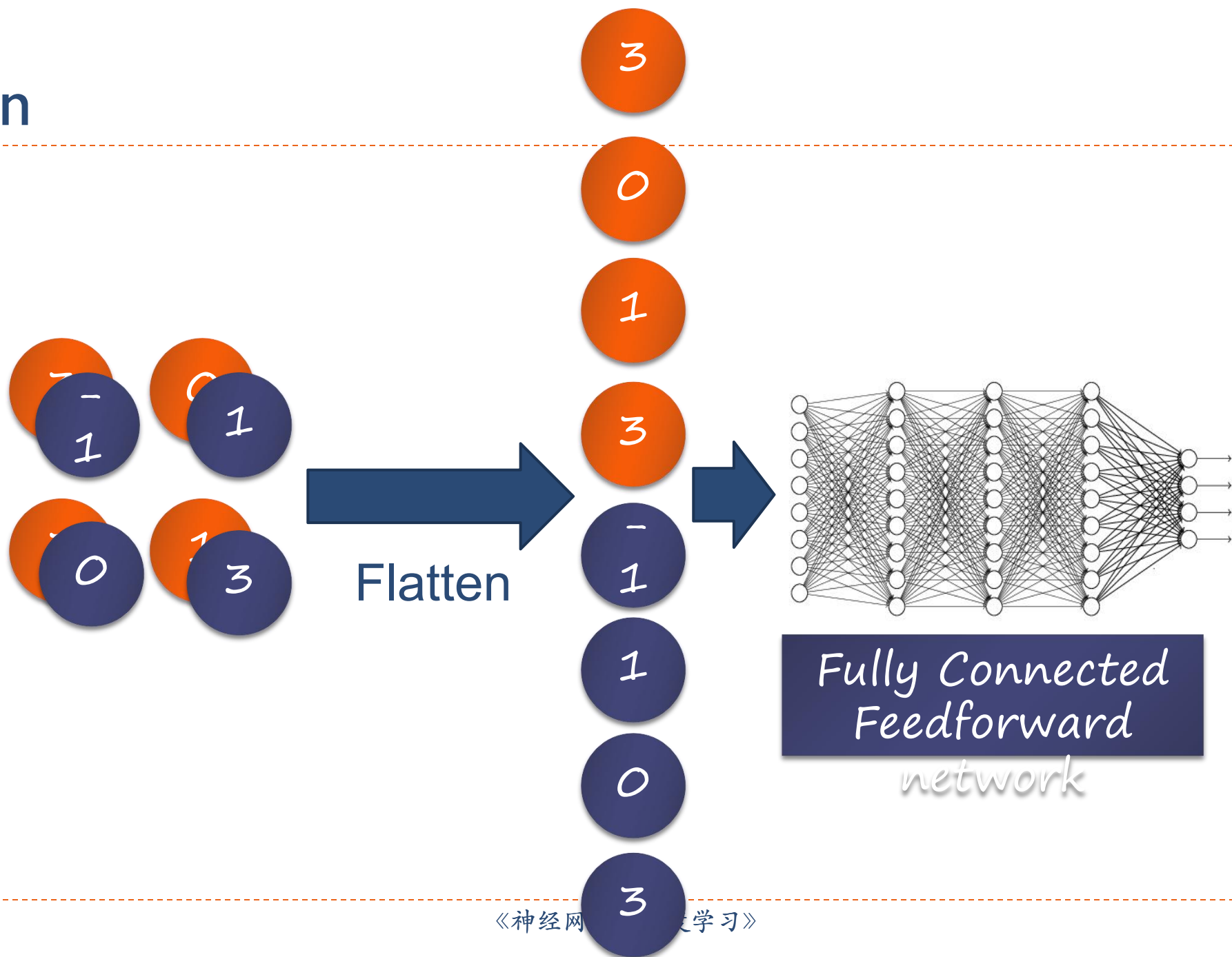


The whole CNN

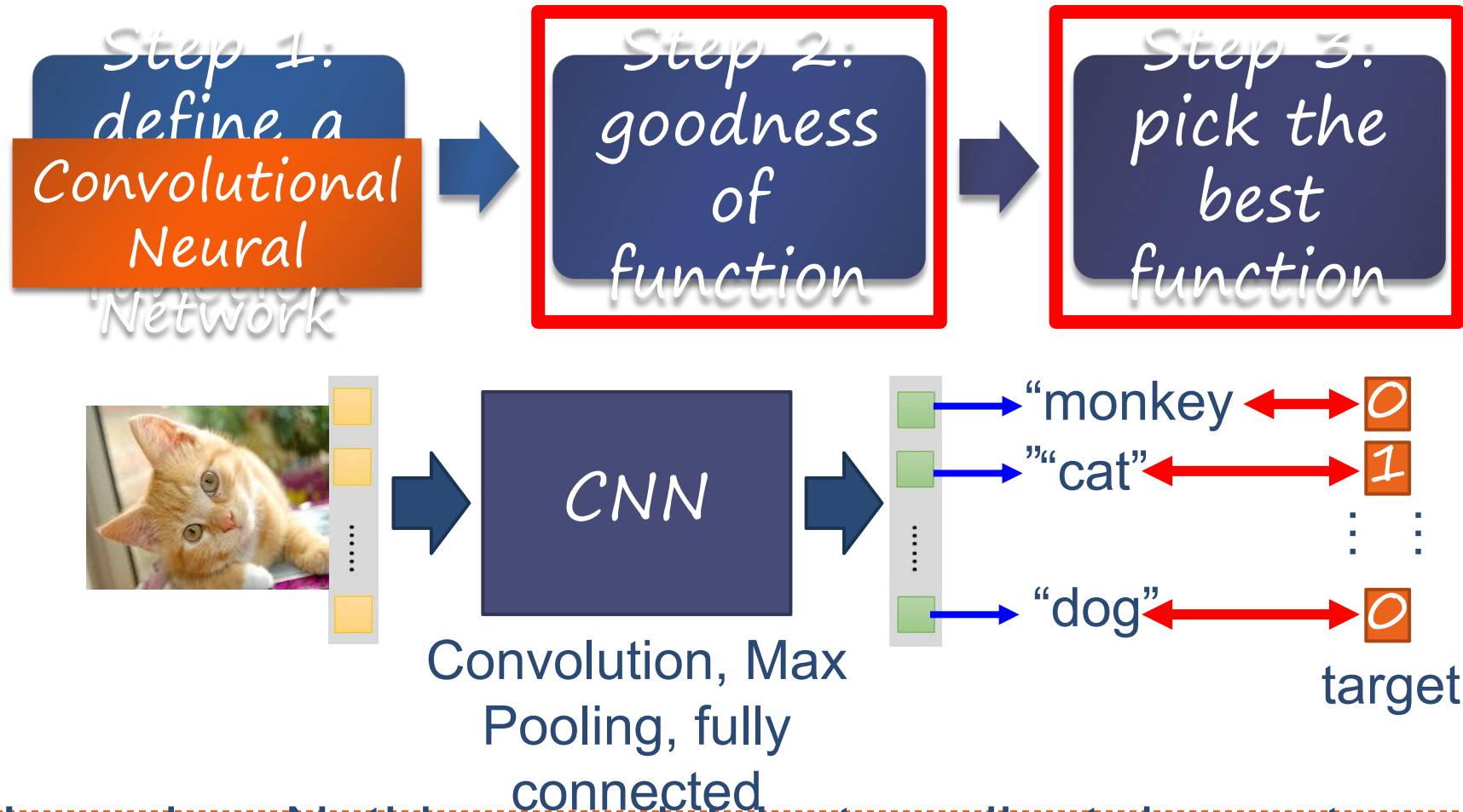
cat dog



Flatten



Convolutional Neural Network

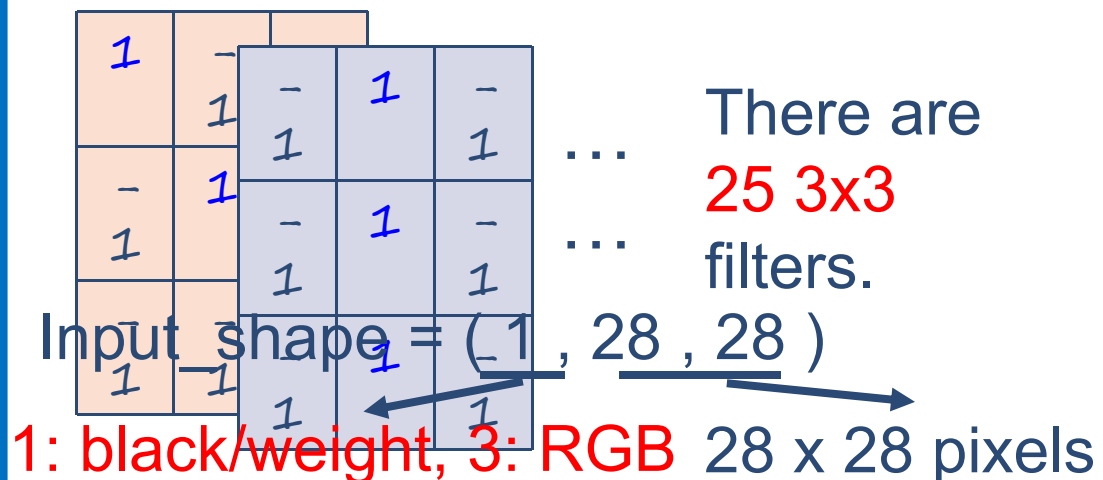


Learning: Nothing special, just gradient descent

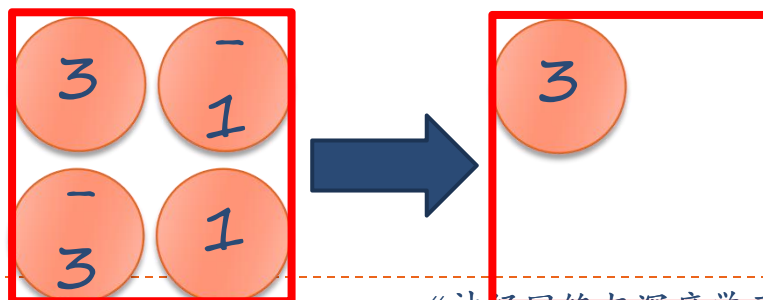
CNN in Keras

Only modified the network structure and input format
(vector $\rightarrow$ 3-D tensor)

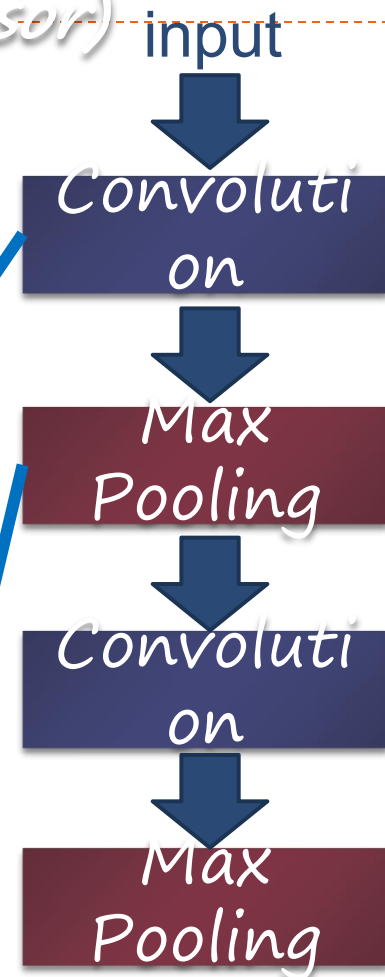
```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(1, 28, 28) ) )
```



```
model2.add( MaxPooling2D( (2, 2) ) )
```



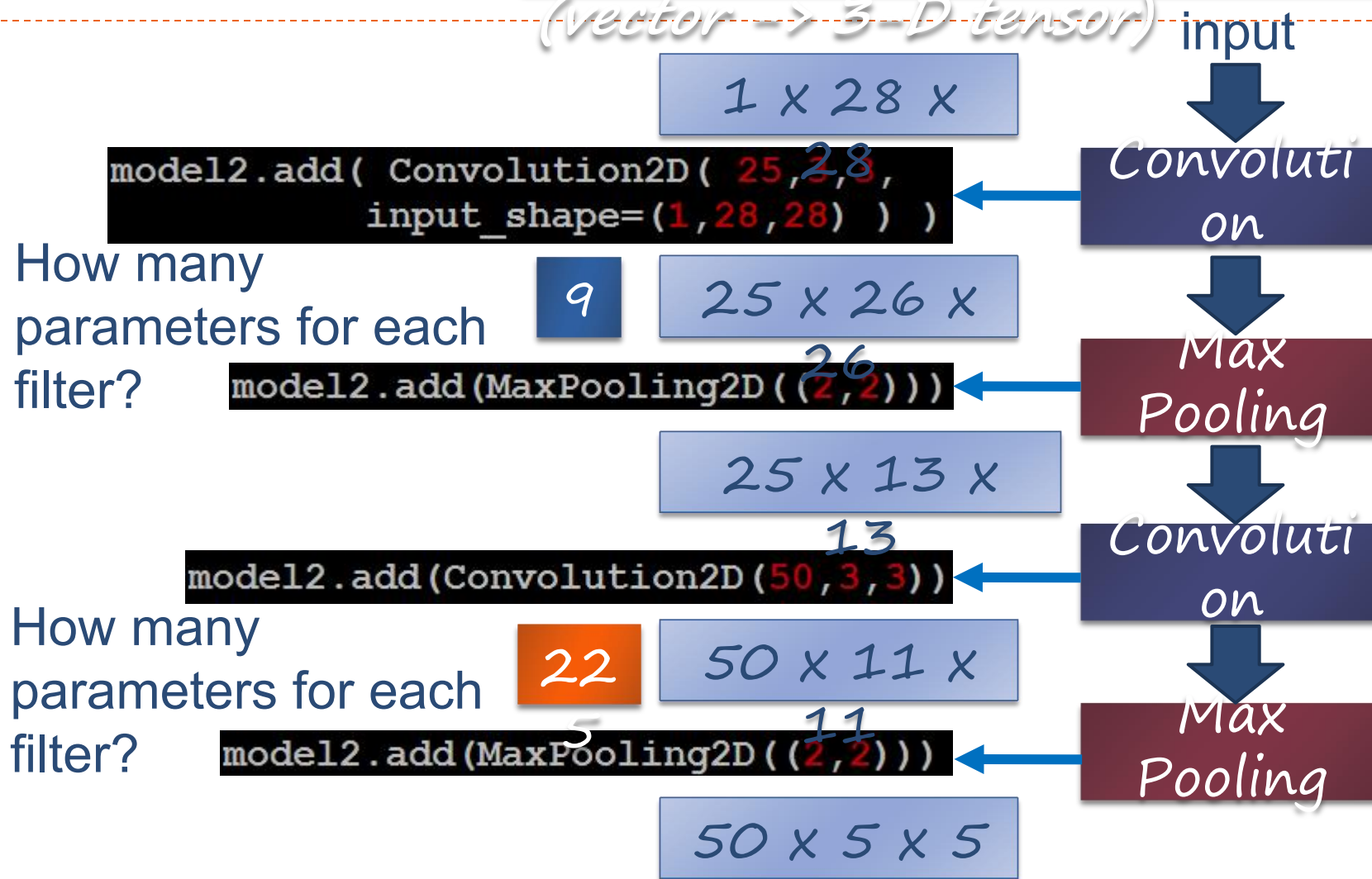
《神经网络与深度学习》



CNN in Keras

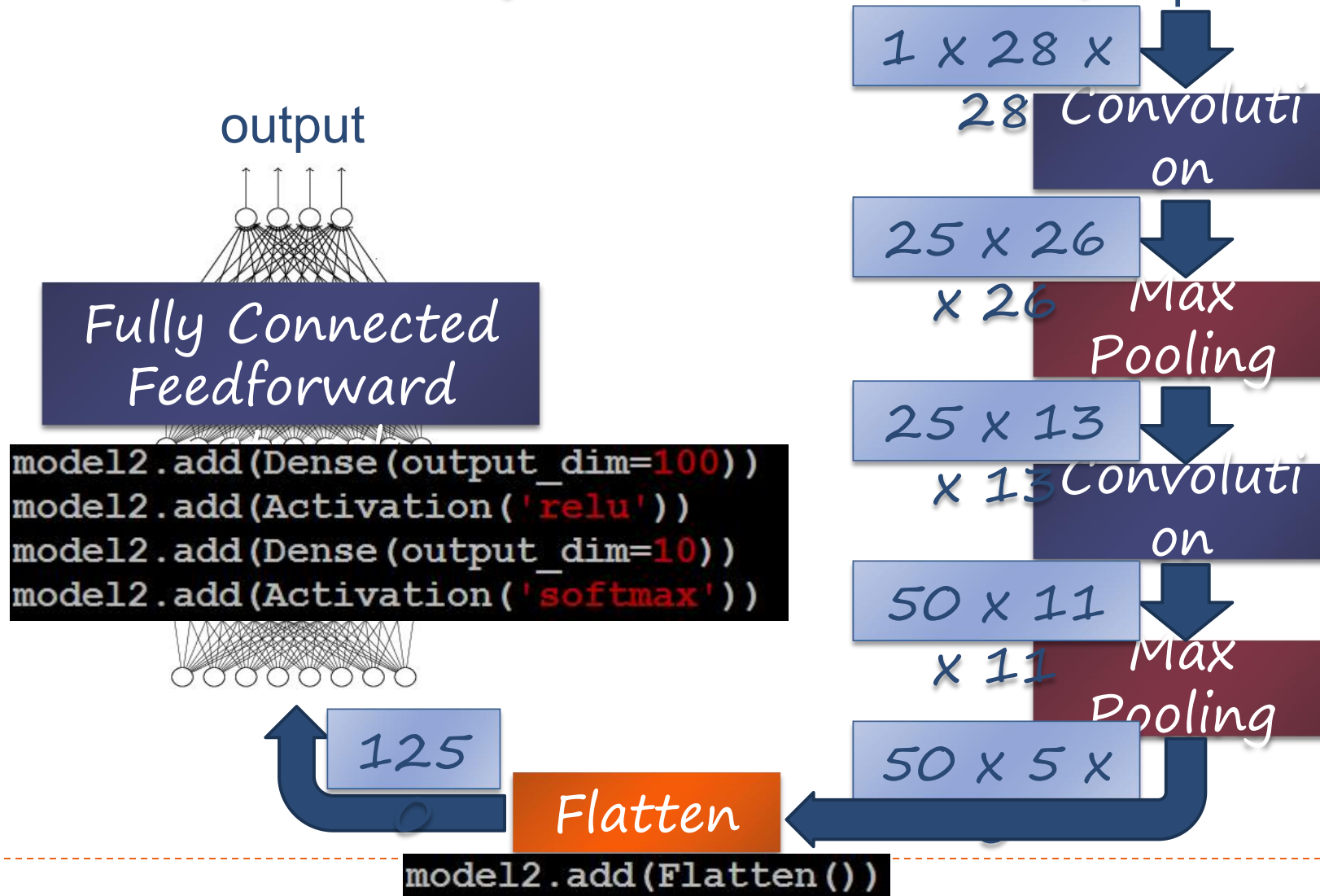
Only modified the network structure and input format

(vector $\rightarrow$ 3-D tensor)

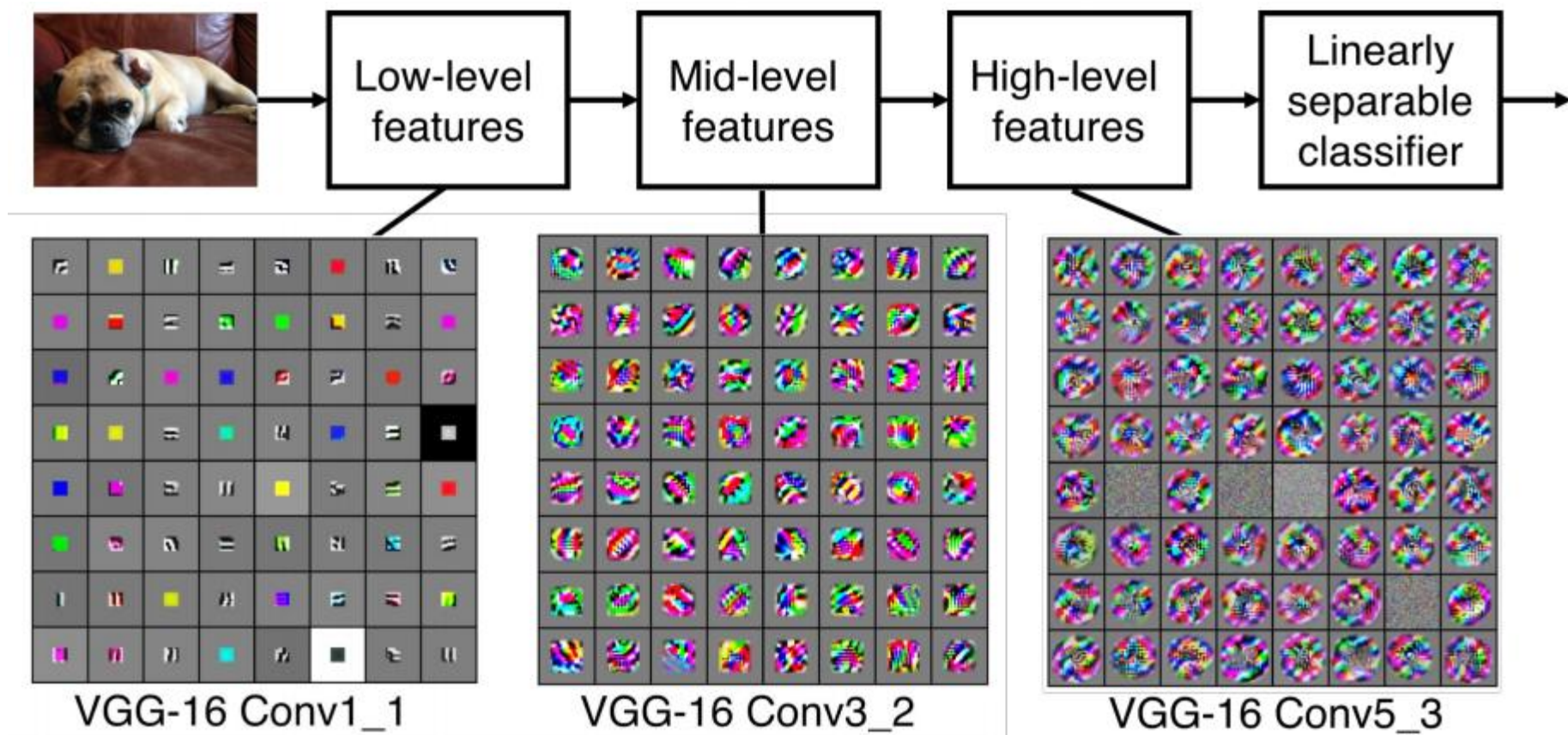


CNN in Keras

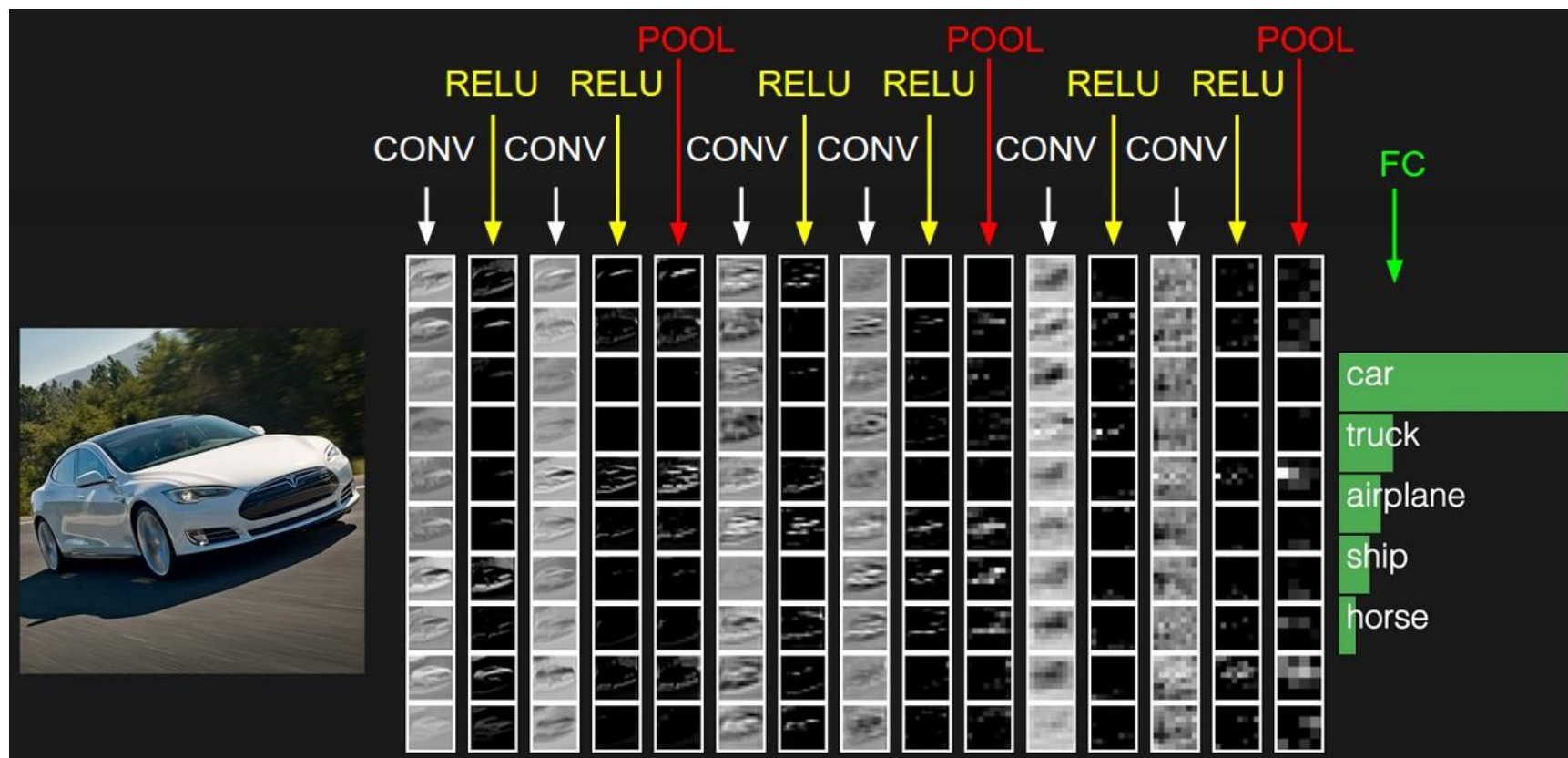
Only modified the network structure and input format
(vector $\rightarrow$ 3-D tensor)



表示学习



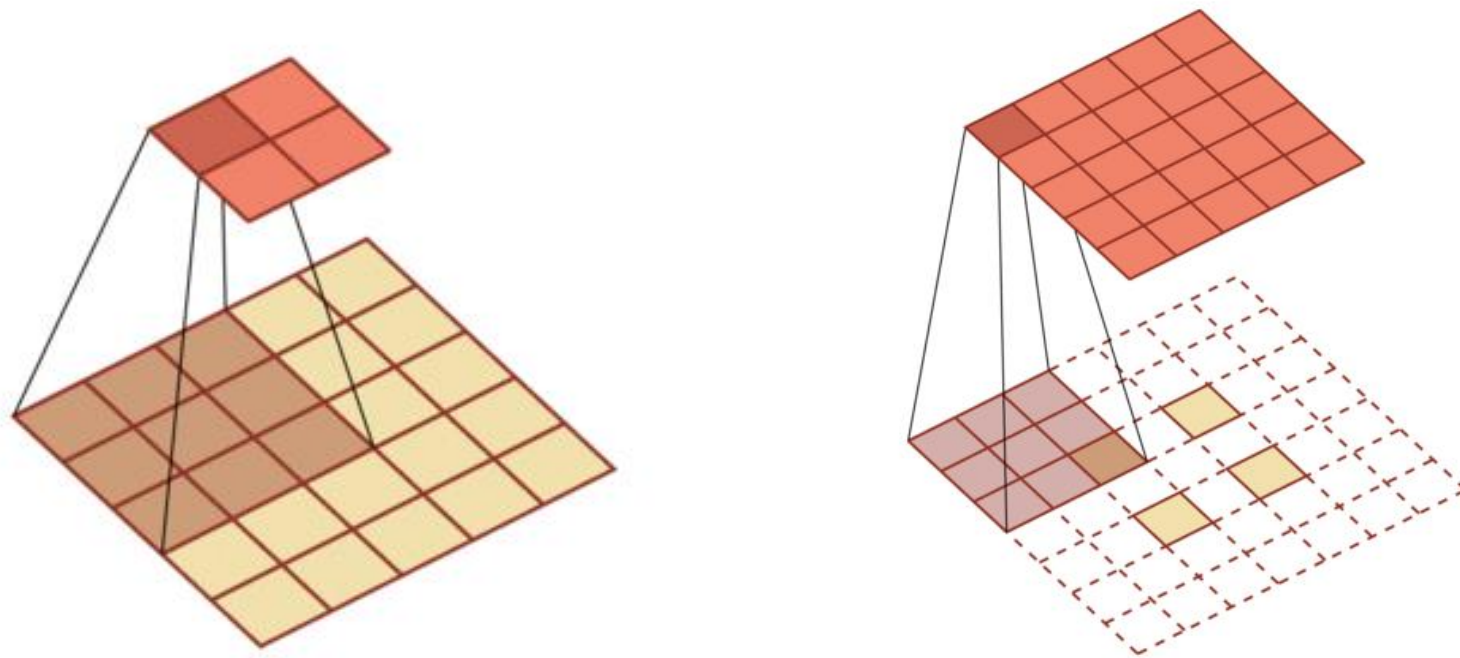
表示学习



其它卷积种类

转置卷积/微步卷积

► 低维特征映射到高维特征



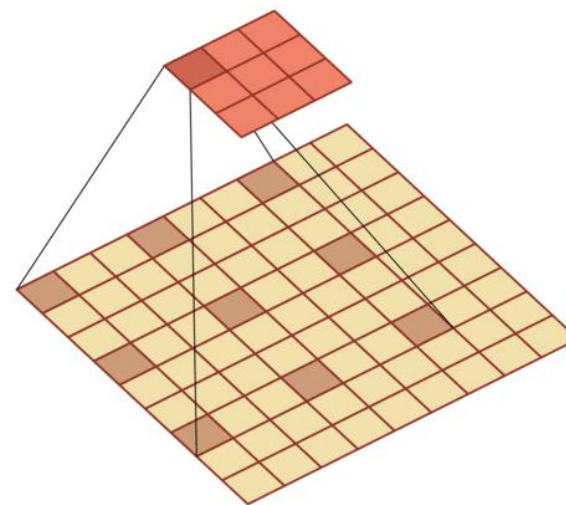
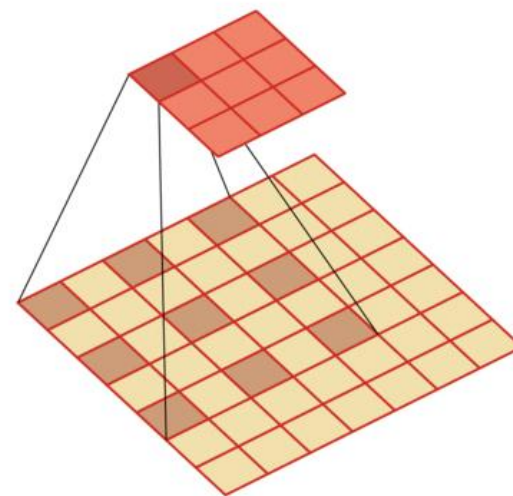
空洞卷积

▶ 如何增加输出单元的感受野

- ▶ 增加卷积核的大小
- ▶ 增加层数来实现
- ▶ 在卷积之前进行汇聚操作

▶ 空洞卷积

- ▶ 通过给卷积核插入“空洞”来变相地增加其大小。

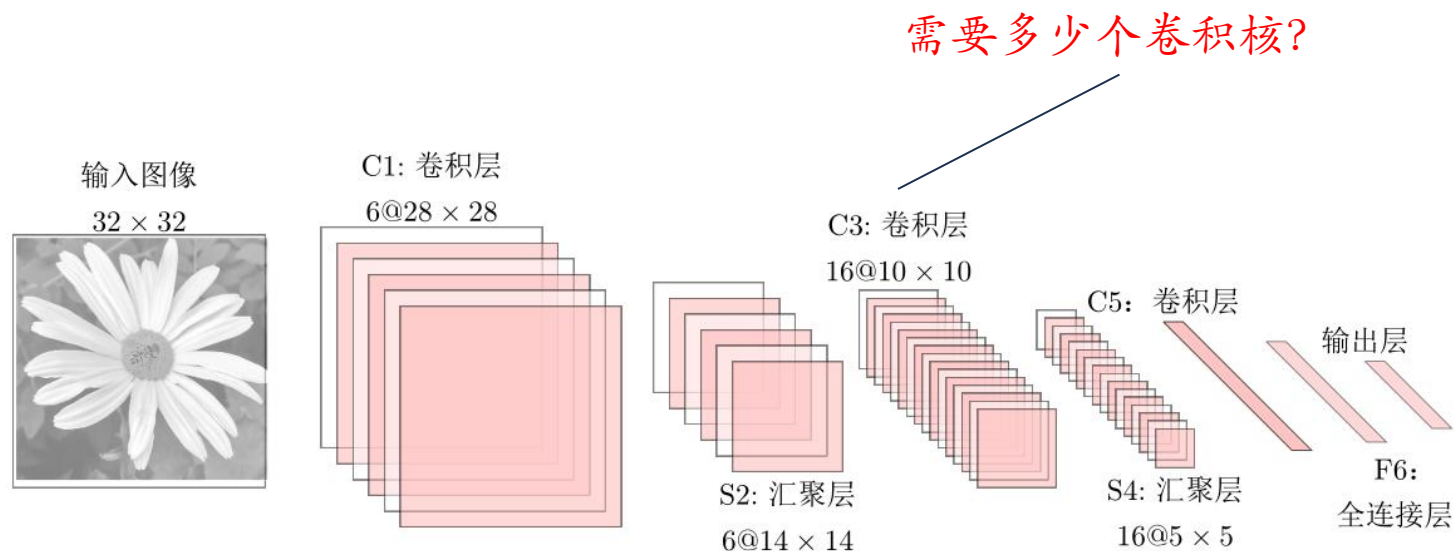


典型的卷积网络

LeNet-5

► LeNet-5 是一个非常成功的神经网络模型。

- 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。
- LeNet-5 共有 7 层。



LeNet-5

$$(M - K + 2P)/S + 1$$

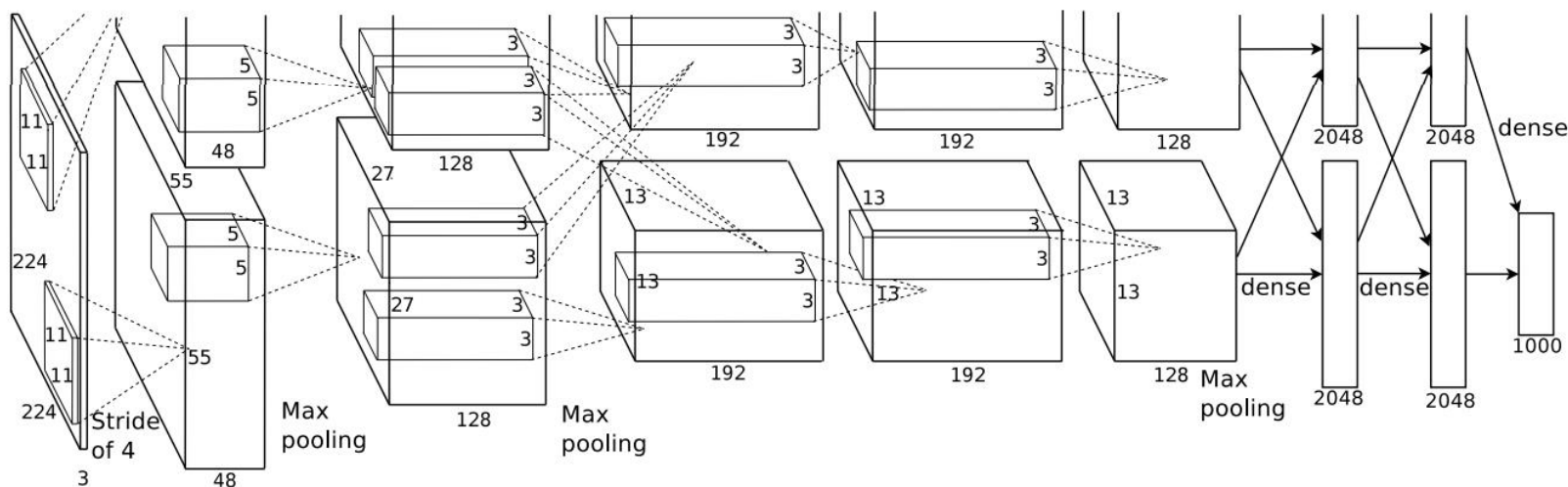
网络层 (操作)	输入	filter	stride	padding	输出	计算公式	参数量
Input	32x32x1				32x32x1		0
Conv1	32x32x1	5x5x6	1	0	28x28x6	5x5x1x6+6	156
MaxPool1	28x28x6	2x2	2	0	14x14x6		0
Conv2	14x14x6	5x5x16	1	0	10x10x16	5x5x6x16+16	2416
MaxPool2	10x10x16	2x2	2	0	5x5x16		0
FC1	5x5x16				120	5x5x16x120+1	48001
FC2	120				84	120x84+1	10081
FC3	84				10	84x10+1	841

▶ 参数总量： 61495 参数内存消耗： 240.214KB

AlexNet

► 2012 ILSVRC winner

- (top 5 error of 16% compared to runner-up with 26% error)
- 第一个现代深度卷积网络模型
 - 首次使用了很多现代深度卷积网络的一些技术方法
 - 使用GPU进行并行训练，采用了ReLU作为非线性激活函数，使用Dropout防止过拟合，使用数据增强
- 5个卷积层、3个汇聚层和3个全连接层



AlexNet

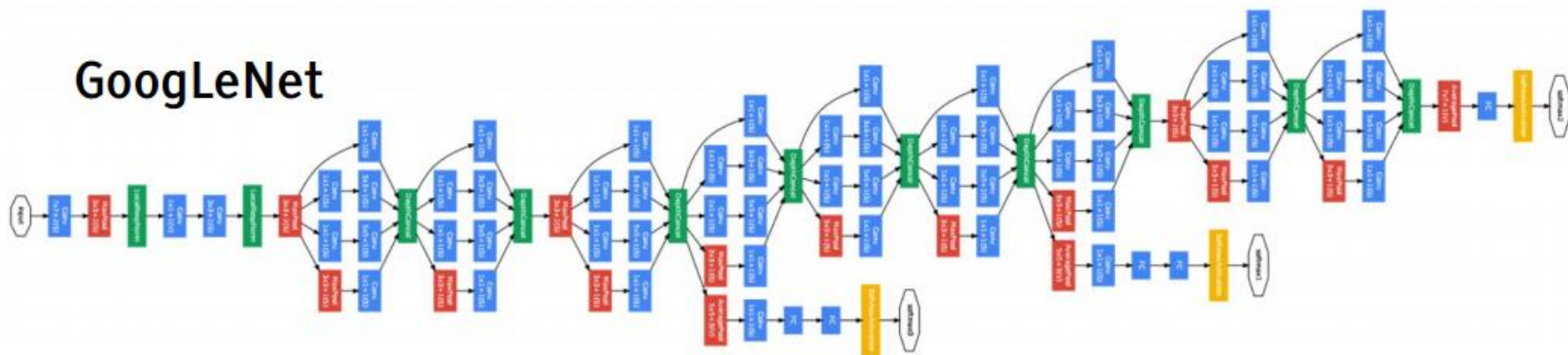
参数总量：
62369155
参数内存消耗：
237.9195MB

网络层 (操作)	输入	filter	stride	padding	输出	计算公式	参数量
Input	227x227x3				227x227x3		0
Conv1	227x227x3	11x11x96	4	0	55x55x96	$11 \times 11 \times 3 \times 96 + 96$	34944
MaxPool1	55x55x96	3x3	2	0	27x27x96		0
Norm1	27x27x96				27x27x96		0
Conv2	27x27x96	5x5x256	1	2	27x27x256	$5 \times 5 \times 96 \times 256 + 256$	614656
MaxPool2	27x27x256	3x3	2	0	13x13x256		0
Norm12	13x13x256				13x13x256		0
Conv3	13x13x256	3x3x384	1	1	13x13x384	$3 \times 3 \times 256 \times 384 + 384$	885120
Conv4	13x13x384	3x3x384	1	1	13x13x384	$3 \times 3 \times 384 \times 384 + 384$	1327488
Conv5	13x13x384	3x3x256	1	1	13x13x256	$3 \times 3 \times 384 \times 256 + 256$	884992
MaxPool3	13x13x256	3x3	2	0	6x6x256		0
FC6	6x6x256				4096	$6 \times 6 \times 256 \times 4096 + 1$	37748737
FC7	4096				4096	$4096 \times 4096 + 1$	16777217
FC8	4096				1000	$4096 \times 1000 + 1$	4096001

Inception网络

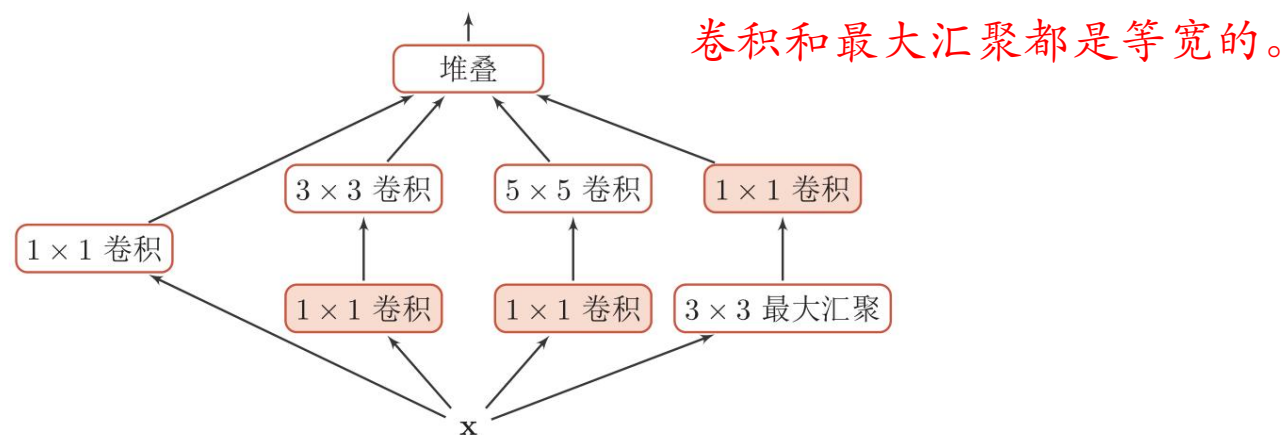
► 2014 ILSVRC winner (22层)

- 参数: GoogLeNet: 4M VS AlexNet: 60M
- 错误率: 6.7%
- Inception网络是由有多个inception模块和少量的汇聚层堆叠而成。



Inception模块 v1

- ▶ 在卷积网络中，如何设置卷积层的卷积核大小是一个十分关键的问题。
- ▶ 在Inception网络中，一个卷积层包含多个不同大小的卷积操作，称为Inception模块。
- ▶ Inception模块同时使用 1×1 、 3×3 、 5×5 等不同大小的卷积核，并将得到的特征映射在深度上拼接（堆叠）起来作为输出特征映射。



Inception模块 v3

- ▶ 用多层的小卷积核来替换大的卷积核，以减少计算量和参数量。
- ▶ 使用两层3x3的卷积来替换v1中的5x5的卷积
- ▶ 使用连续的nx1和1xn来替换nxn的卷积。

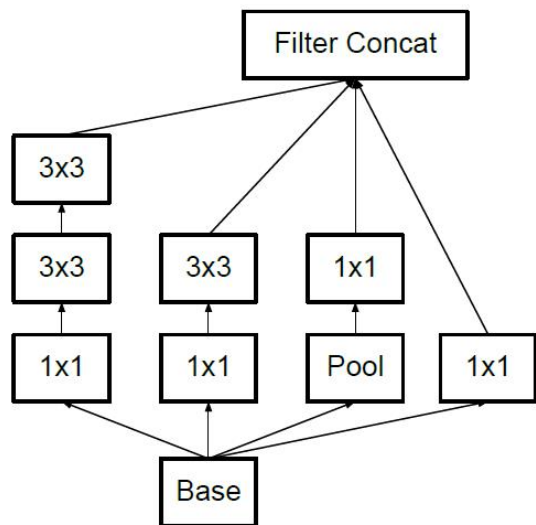


Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle [3] of Section [2].

<http://blog.csdn.net/xbinworld>

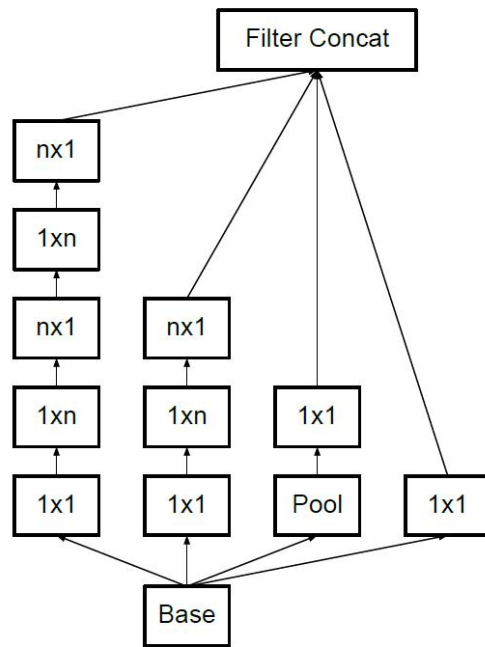
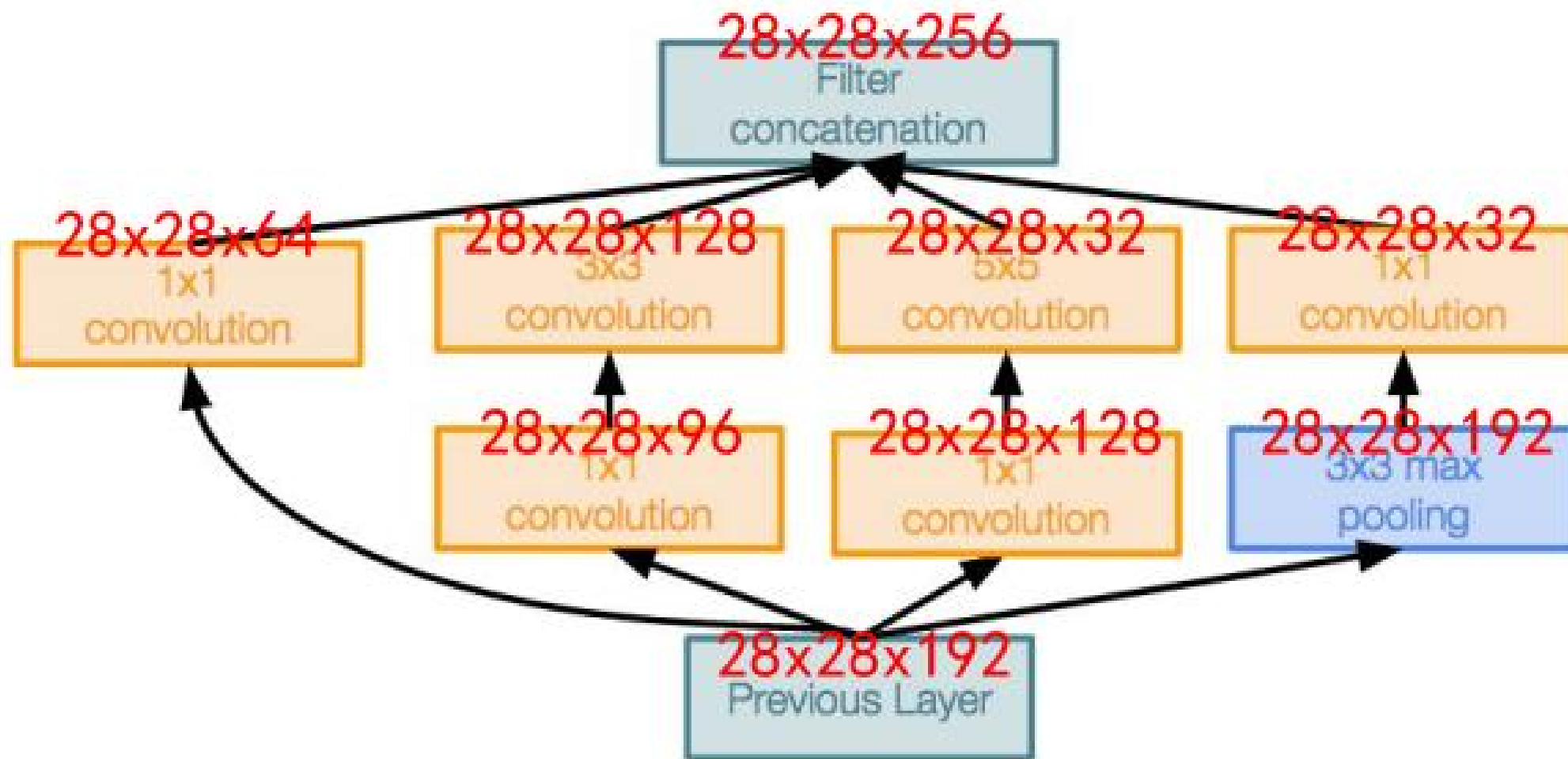


Figure 6. Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle [3])

Inception模块

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								



Inception module

<http://blog.csdn.net/qian99>

VGG 网络原理

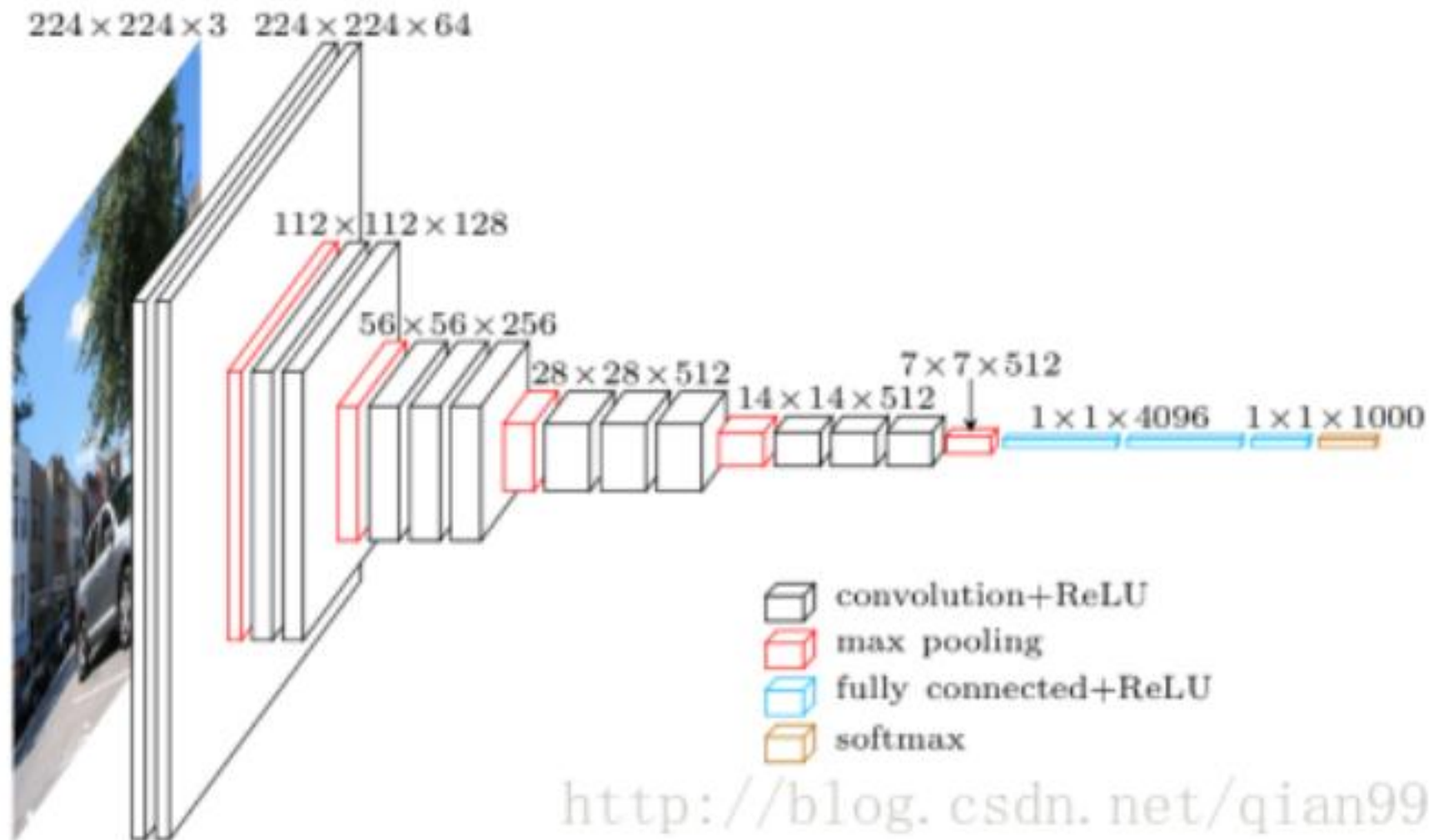
- ▶ VGG是Oxford的Visual Geometry Group组提出。VGG网络探索了提升网络的深度对最终的图像识别准确率的重要性，同时在VGG中尝试使用小的卷积核来构建深层的卷积网络。VGG在当年的ILSVRC 2014上取得了第一的成绩，证明了增加网络的深度能够在一定程度上提高网络的性能。

VGG原理

- ▶ VGG网络有两种结构，分别是VGG16和VGG19，两者只是网络深度不一样。以VGG16为例，在VGG16中，相比于AlexNet，最大改进是采用几个 3×3 的卷积核代替了AlexNet中的较大的卷积核，如： 11×11 ， 5×5 等。
- ▶ 相比于大的卷积核，采用多个较小卷积核的堆叠，
 - ▶ 一方面能通过多层的非线性特性增强模型的学习能力（卷积+ReLU为一个组合）
 - ▶ 能进一步减少模型的参数。

VGG(16/19 16为例)

参数总量:
138357544
138348355
参数内存消耗:
527.7570MB



残差网络

- ▶ 在VGG中，卷积网络达到了19层，在GoogLeNet中，网络史无前例的达到了22层。网络的精度会随着网络的层数增多而增多吗？在深度学习中，网络层数增多一般会伴着下面几个问题
 - ▶ 计算资源的消耗
 - ▶ 模型容易过拟合
 - ▶ 梯度消失/梯度爆炸问题

残差网络

- ▶ 计算资源的消耗:
- ▶ 模型容易过拟合
- ▶ 梯度消失/梯度爆炸问题

问题1可以通过GPU集群来解决;

问题2可通过采集海量数据, 配合Dropout正则化等方法也可以有效避免;

问题3通过Batch Normalization也可以避免。

只要无尽的增加网络的层数, 我们就能从此获益??

残差网络

- ▶ **现实：**
- ▶ 随着网络层数的增加，**网络退化**：随着网络层数的增多，训练集loss值逐渐下降，然后趋于饱和，如果再增加网络深度，训练集loss反而会增大。
- ▶ 上述现象并不是**过拟合**，因为在过拟合中训练loss是一直减小的。
- ▶ 当网络退化时，浅层网络能够达到比深层网络更好的训练效果，这时如果我们把低层的特征传到高层，那么效果应该至少不比浅层的网络效果差，

残差网络

- ▶ 如果一个VGG-100网络在第98层使用的是和VGG-16第14层一模一样的特征，那么VGG-100的效果应该会和VGG-16的效果相同。
- ▶ 所以，我们可以在VGG-100的98层和14层之间添加一条直接映射（Identity Mapping）来达到此效果。
- ▶ 从信息论的角度讲，由于DPI（数据处理不等式）的存在，在前向传输的过程中，随着层数的加深，Feature Map包含的图像信息会逐层减少，而ResNet的直接映射的加入，保证了1+1层的网络一定比1层包含更多的图像信息。
- ▶ **残差网络本质：使用直接映射来连接网络不同层。**

残差网络

- ▶ 残差网络（Residual Network, ResNet）是由来自Microsoft Research的4位学者（何恺明、张祥雨、任少卿、孙剑）提出的卷积神经网络，ResNet 在 2015 年的 ILSVRC（ImageNet Large Scale Visual Recognition Challenge）中取得了图像分类、检测、定位三个冠军。
- ▶ 2016 年 CVPR 论文：《Deep Residual Learning for Image Recognition》就介绍了 ResNet，该论文截至当前 (2026.5.14) 已被引用超过 30 万次（**平均每年两万余次的增幅**）。

残差网络

▶ 截至**2025年8月**的最新公开资料，全球引用率最高的十位学者，其引用数据来源于**AD Scientific Index**等权威学术排名平台。

Yoshua Bengio，1964年，62岁

- 总被引量：约**97.4**万次
- 身份：深度学习三巨头之一、图灵奖得主、蒙特利尔大学教授
- 标志性成果：生成对抗网络（GAN）、《Deep Learning》经典教材
- 近五年被引：698,008次

排名：第一

Geoffrey Hinton，1947年，78岁

- 总被引量：约**95**万次
- 身份：“AI教父”、2024年诺贝尔物理学奖得主、谷歌DeepMind杰出科学家
- 标志性成果：反向传播算法、AlexNet（2012年ImageNet突破）
- 近五年被引：57万+次

排名：第二

何恺明（Kaiming He），1984年，42岁

- 总被引量：超**73**万次
- 身份：计算机视觉专家、ResNet提出者
- 标志性成果：ResNet（21世纪单篇被引最多论文，超29万次）

排名：第五

残差网络

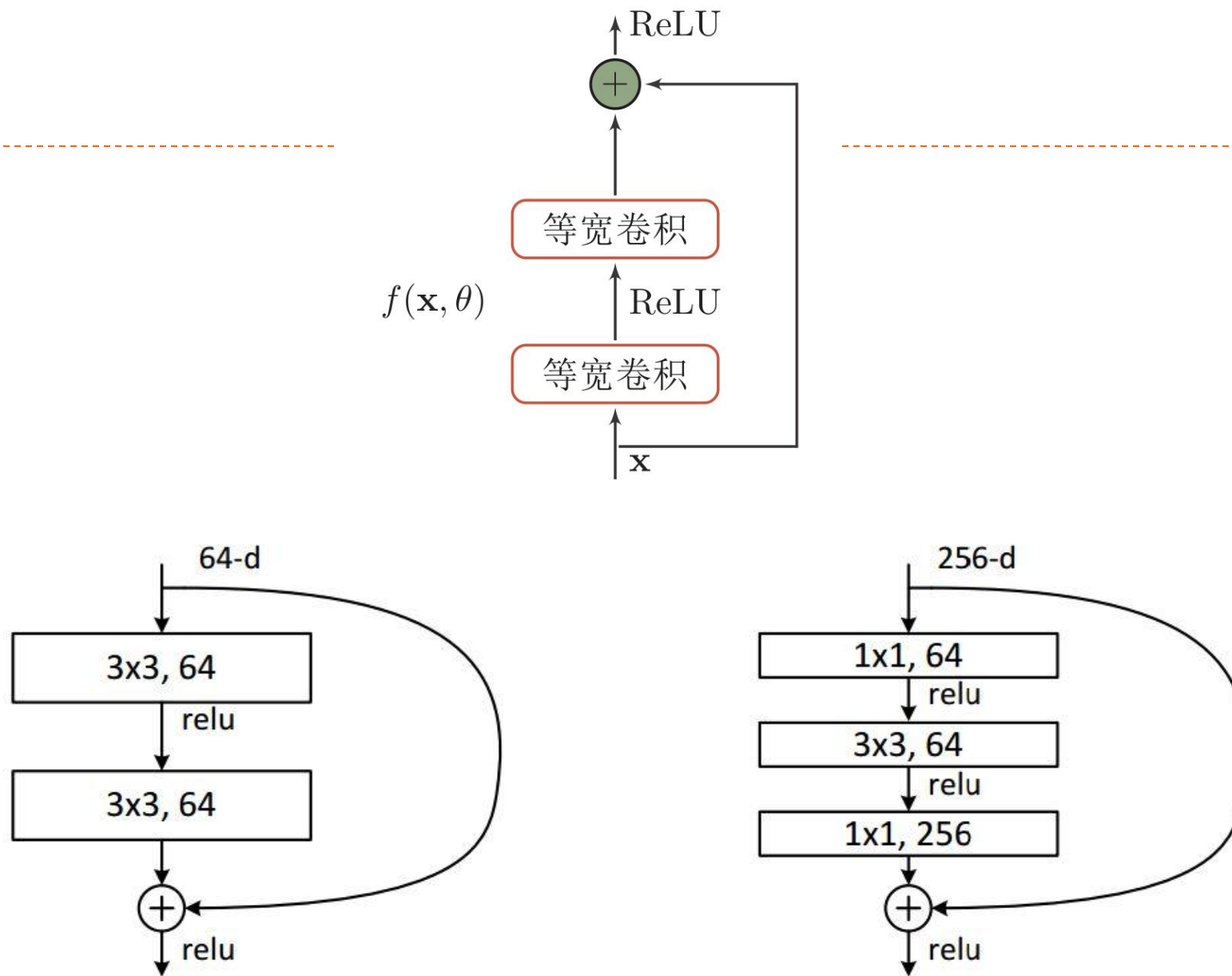
- ▶ 残差神经网络的主要贡献是发现了“退化现象 (Degradation)”，并针对退化现象发明了“快捷连接 (Shortcut connection)”（或者跳过连接），极大的消除了深度过大的神经网络训练困难问题。神经网络的“深度”首次突破了 100 层、最大的神经网络甚至超过了 1000 层。

残差网络

- ▶ 残差网络（Residual Network, ResNet）是通过给非线性的卷积层增加直连边的方式来提高信息的传播效率。
- ▶ 假设在一个深度网络中，我们期望一个非线性单元（可以为一层或多层的卷积层） $f(\mathbf{x}, \theta)$ 去逼近一个目标函数为 $h(\mathbf{x})$ 。
- ▶ 将目标函数拆分成两部分：恒等函数和残差函数

$$h(\mathbf{x}) = \underbrace{\mathbf{x}}_{\text{恒等函数}} + \underbrace{(h(\mathbf{x}) - \mathbf{x})}_{\text{残差函数}} \rightarrow f(\mathbf{x}, \theta)$$

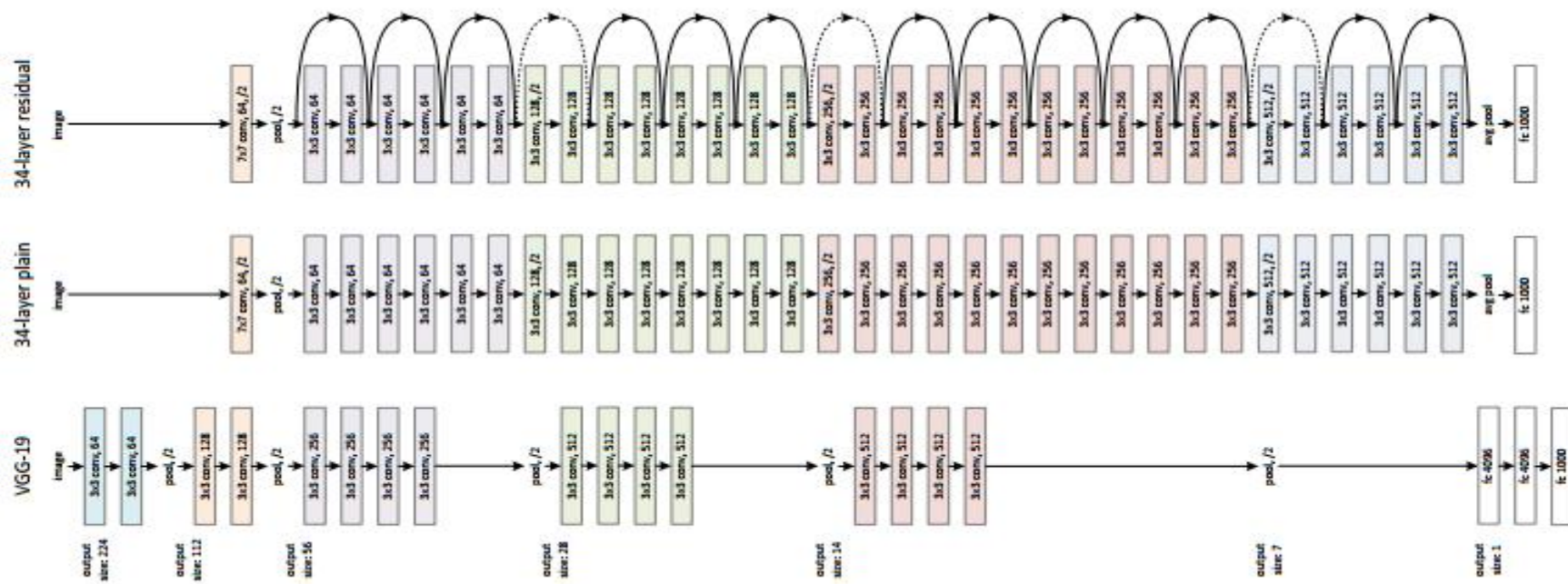
残差单元



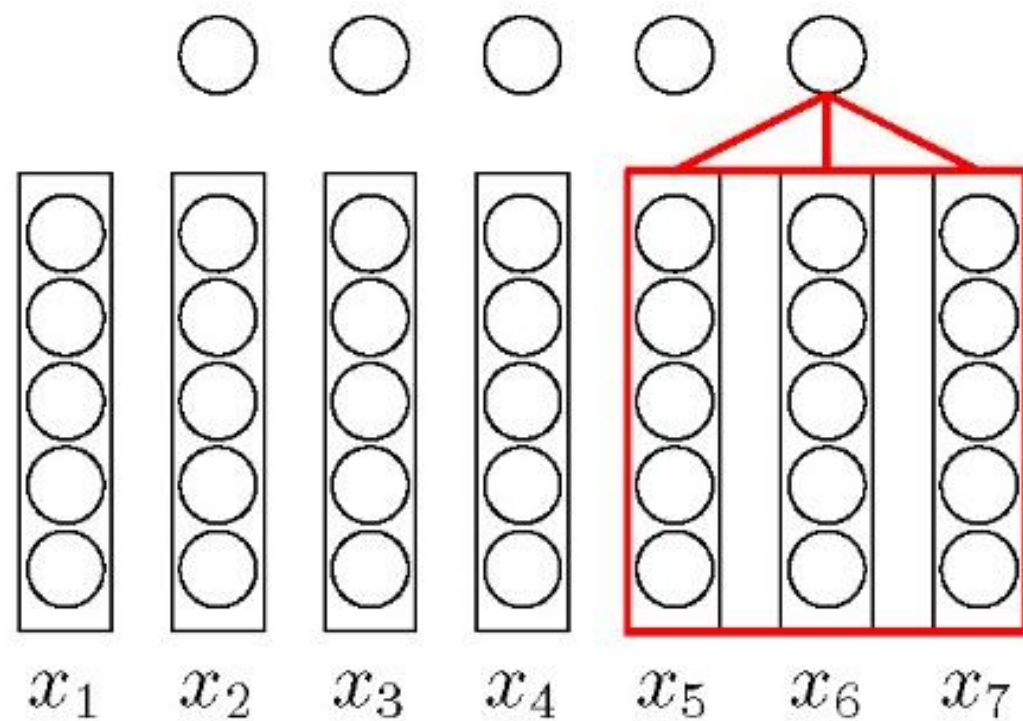
ResNet

► 2015 ILSVRC winner (152层)

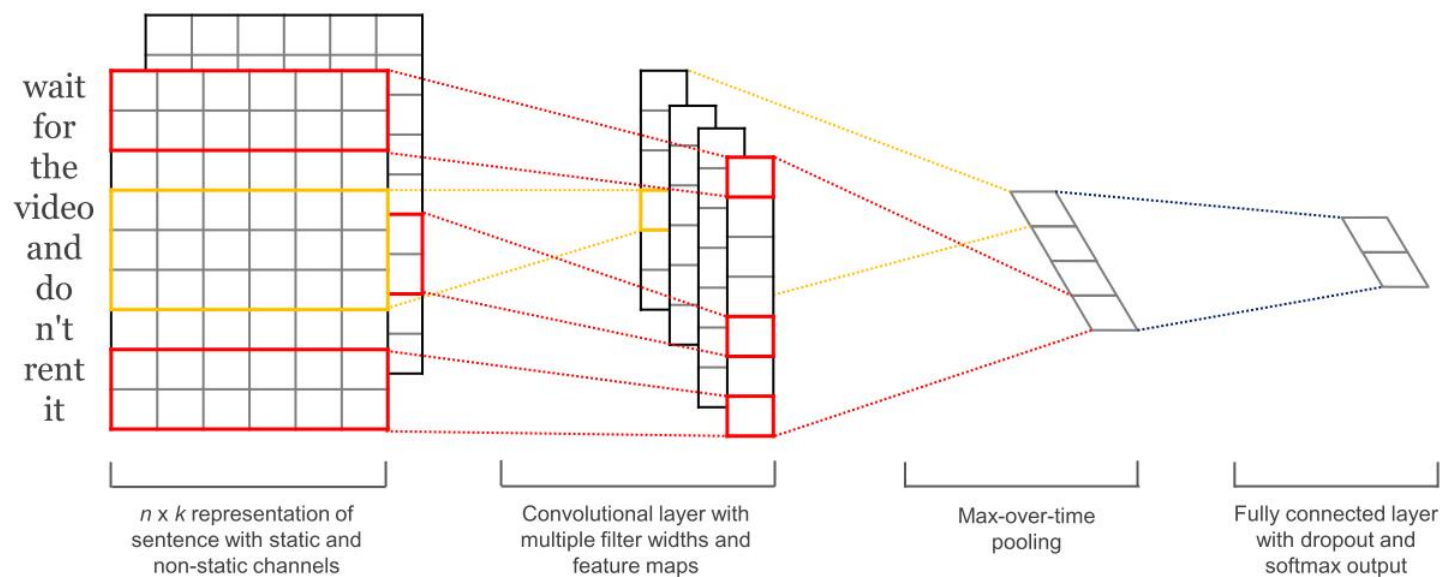
► 错误率: 3.57%



文本序列的卷积

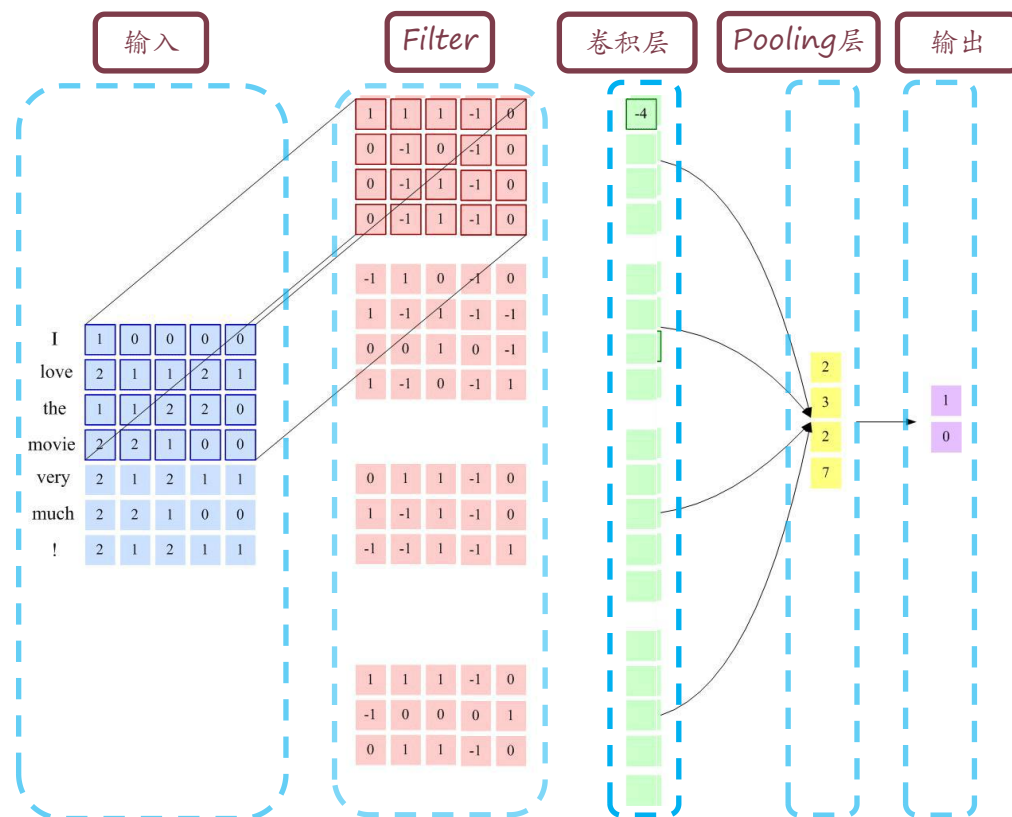


基于卷积模型的句子表示



Y. Kim. "Convolutional neural networks for sentence classification" . In: *arXiv preprint arXiv:1408.5882* (2014).

文本序列的卷积模型



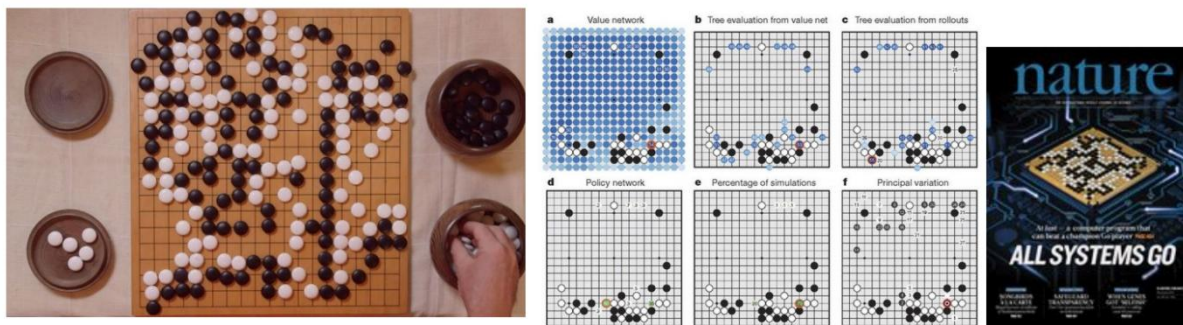
CNN 可视化：滤波器

► AlexNet 中的滤波器 (96 filters [11x11x3])



卷积的应用

AlphaGo



The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

policy network:

[19x19x48] Input

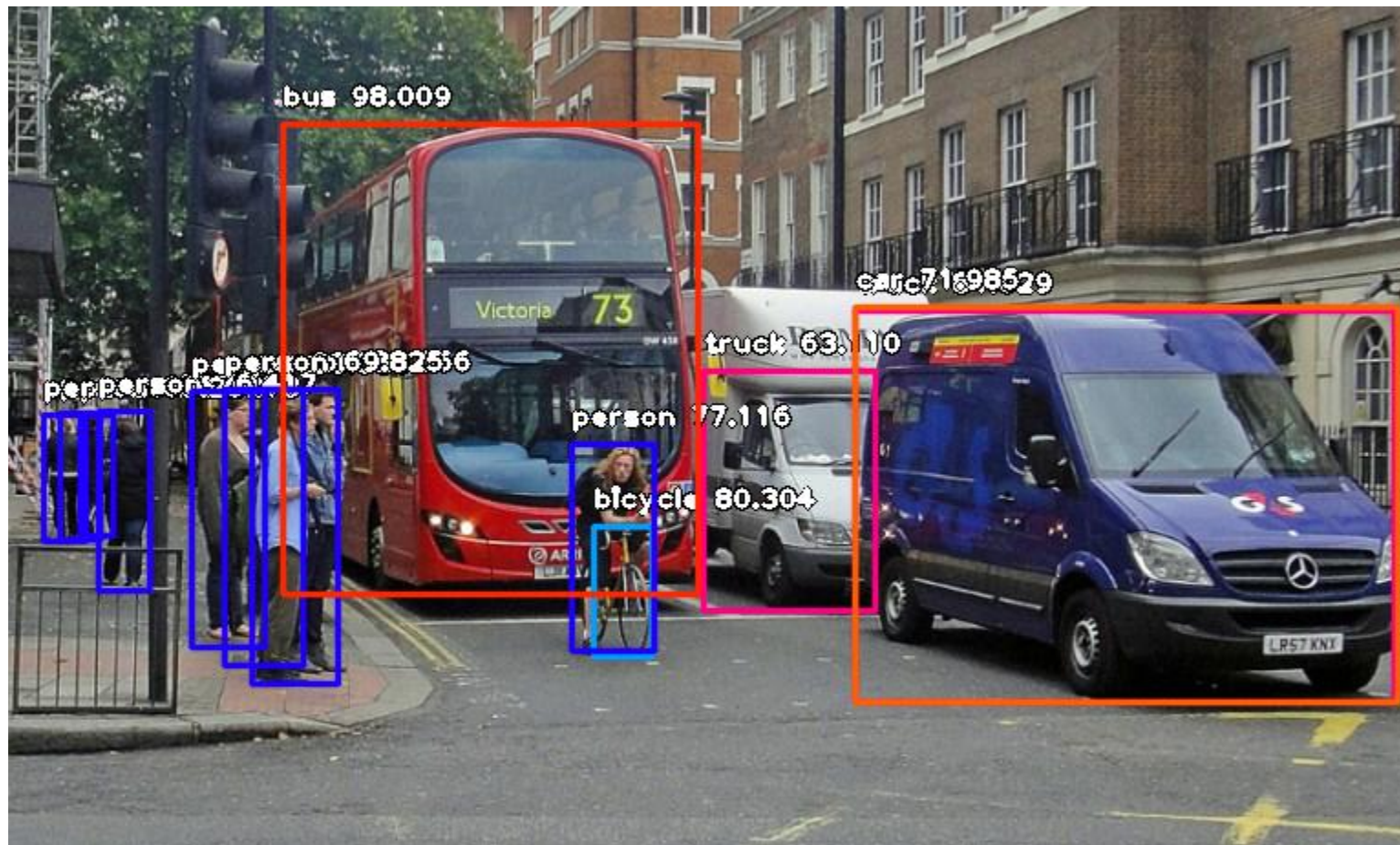
CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]

CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)

- ▶ 分布式系统: 1202 个CPU 和176 块GPU
- ▶ 单机版: 48 个CPU 和8 块GPU
- ▶ 走子速度: 3 毫秒-2 微秒

目标检测 (Object Detection)



Mask RCNN

Mask R-CNN是一个实例分割（Instance segmentation）算法，可以用来做“目标检测”、“目标实例分割”、“目标关键点检测”。

Mask R-CNN是一个非常灵活的框架，可以增加不同的分支完成不同的任务，可以完成目标分类、目标检测、语义分割、实例分割、人体姿势识别等多种任务

Mask RCNN

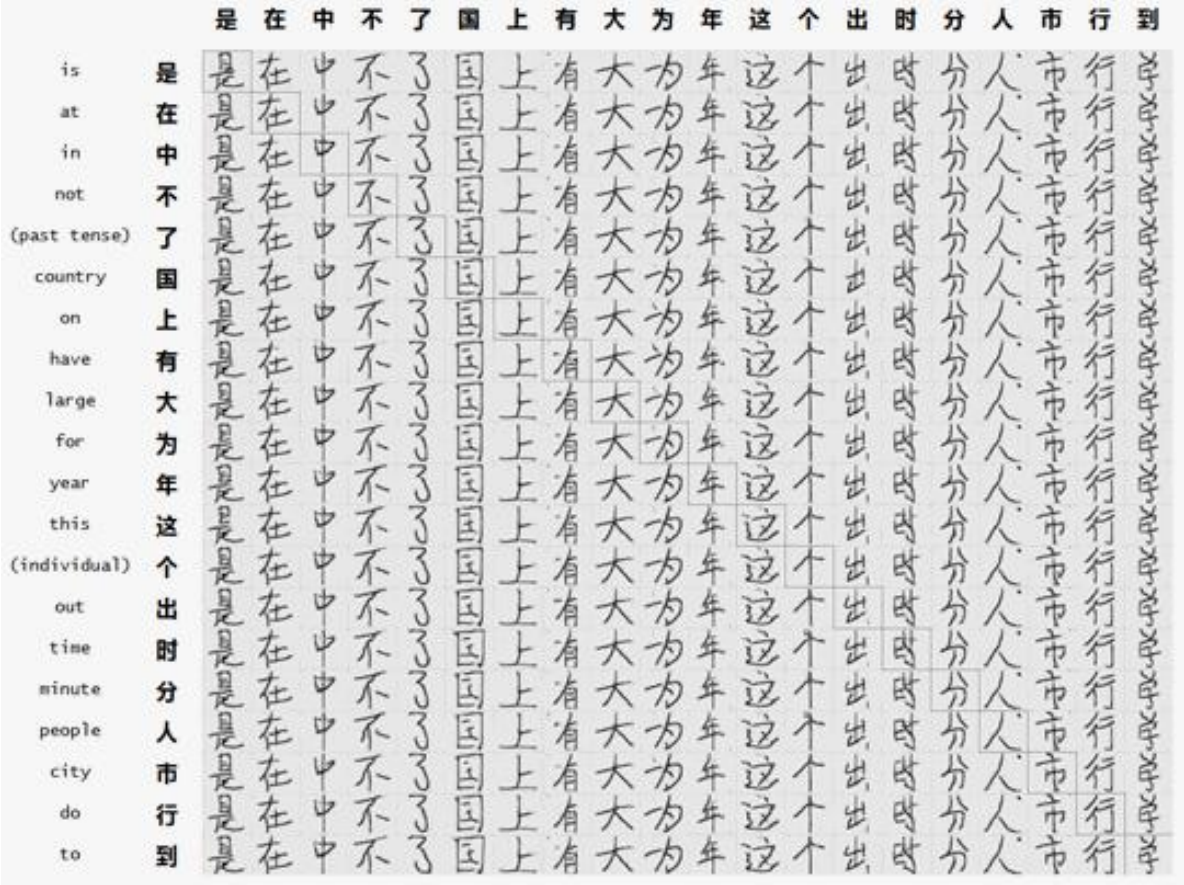


Figure 4. More results of **Mask R-CNN** on COCO test images, using ResNet-101-FPN and running at 5 fps, with 35.7 mask AP (Table 1).

OCR (电子设备上的数字识别)



图像生成



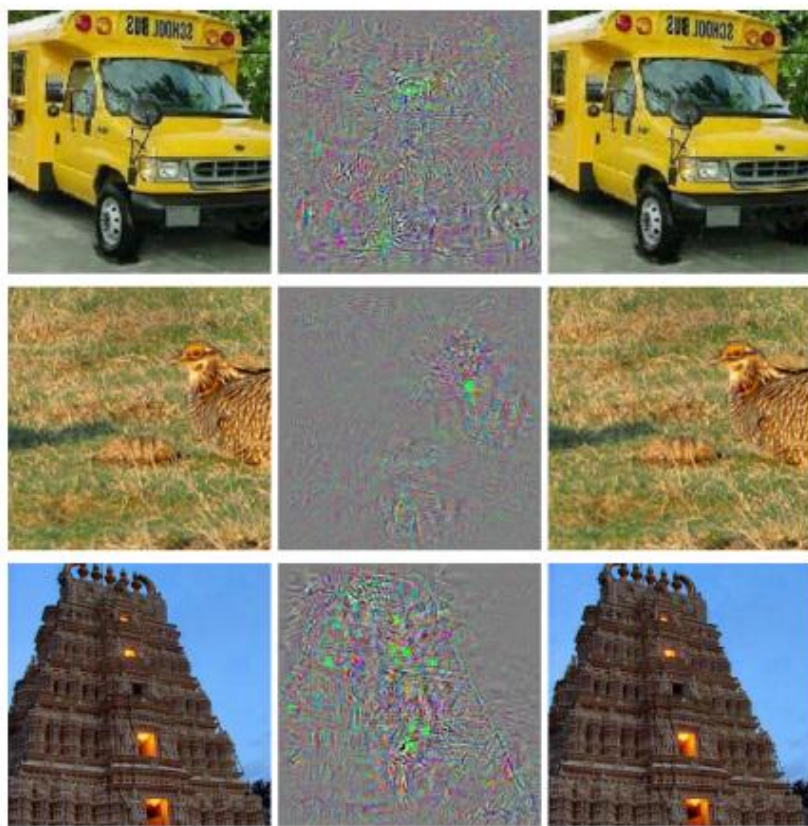
Deep Dream



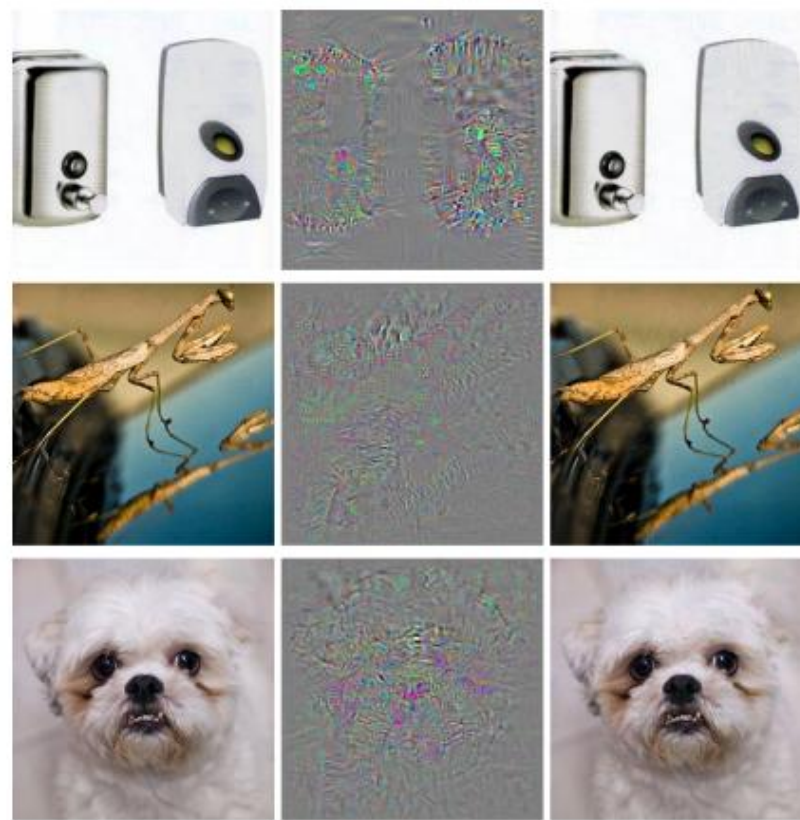
画风迁移



对抗样本



(a)



(b)

课后作业

▶ CNN的局部性假设合理吗？

▶ 如何改进？

▶ 编程练习

▶ <https://github.com/nndl/exercise/>

▶ [chap5_CNN](#)

▶ 图像分类

谢 谢

<https://nndl.github.io/>